Intruder Alert System Through Wireless Sensor Network

A PROJECT REPORT

Submitted by

SANIDHYA BHAGAT [RA2111003011522]
SHAILESH RAJ [RA2111003011479]

Under the Guidance of

Dr. P. MADHAVAN

Professor, Department of Computing Technologies

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE ENGINEERING

DEPARTMENT OF COMPUTING TECHNOLOGIES

COLLEGE OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE ANDTECHNOLOGY

KATTANKULATHUR- 603 203

MAY 2025

SRM Institute of Science & Technology Own Work Declaration Form

Degree/ Course: Bachelor of Technology, Computer Science Engineering

Student Names**:** Sanidhya Bhagat, Shailesh Raj

Registration Numbers: RA2111003011522, RA211103011479

Title of Work: Intruder Alert System Through Wireless Sensor Network

We hereby certify that this assessment compiles with the University's Rules and Regulations relating to Academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

We confirm that all the work contained in this assessment is our own except where indicated, and that We have met the following conditions:

- Clearly references / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not my own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that I have received from others (e.g. fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook / University website

We understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

DECLARATION:	
We are aware of and understand the University's policy on Academic misconduct and plagiarism and we certify that this assessment is our own work, except were indicated by referring, and that We have followed the good academic practices noted above.	
RA2111003011511	**RA2111003011479**

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Under Section 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that 18CSP109L / I8CSP111L project report titled "INTRUDER ALERT SYSTEM THROGH WIRELESS SENSOR NETWORK" is the Bonafide work of **SHAILESH RAJ[RA2111003011479], SANIDHYA BHAGAT [RA2111003011522]** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion or on this or any other candidate.

SIGNATURE

Dr. P. MADHAVAN
Supervisor
Professor
Dept. of Computing
Technologies

SIGNATURE

Dr. G. NIRANJANA
Professor & Head
Dept. of
Computing Technologies

Examiner 1

Examiner 2

ACKNOWLEDGEMENTS

We express our humble gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to **Dr. Leenus Jesu Martin M,** Dean-CET, SRM Institute of Science and Technology, for his invaluable support.

We wish to thank **Dr. Revathi Venkataraman**, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

We encompass our sincere thanks to **Dr. M. Pushpalatha**, Professor and Associate Chairperson, School of Computing and **Dr. C. Lakshmi,** Professor and Associate Chairperson, School of Computing, SRM Institute of Science and Technology, for their invaluable support.

We are incredibly grateful to our Head of the Department, **Dr. G. Niranjana**, Professor, Department of Computing Technologies, SRM Institute of Science and Technology, for her suggestions and encouragement a tall the stages of the project work.

We want to convey our thanks to our Project Coordinators**, Dr.R.Vidhya, Dr.M.Arul Prakash and Dr.M.Revathi** and Panel Head, **Dr. P. Madhavan** and Panel Members, **Mr. KC. Prabu Shankar**, Assistant Professor and **Dr. M. Murali**, Professor, Department of Computing Technologies, SRM Institute of Science and Technology, for their inputs during the project reviews and support.

We register our immeasurable thanks to our Faculty Advisor, **Dr. Sibi Amaran**, Department of Computing Technologies, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to our guide, **Dr. P. Madhavan,** Department of Computing Technologies, S.R.M Institute of Science and Technology, for providing us with an opportunity to pursue our project under her mentorship. She provided us with freedom and support to explore the research topics of our interest. Her passion for solving problems and making a difference in the world has always been inspiring.

We sincerely thank all the staff and students of Computing Technologies, School of Computing, S.R.M Institute of Science and Technology, for their help during our project. Finally, we would like to thank our parents, family members, and friends for their unconditional love, constant support and encouragement

Sanidhya Bhagat [RA2111003011522]
Shailesh Raj [RA2111003011479]

ABSTRACT

This report presents the comprehensive design and implementation of an advanced alert system for Wireless Sensor Networks (WSNs), aimed at enhancing the reliability, accuracy, and efficiency of real-time monitoring applications. Traditional WSNs encounter significant challenges related to energy consumption, data transmission reliability, and scalability, often limiting their applicability in critical scenarios that demand prompt and accurate responses. The proposed system integrates a hierarchical network architecture with energy-efficient algorithms and robust communication protocols to overcome these challenges. The alert system can detect critical events and transmit alerts through a multi-tiered, redundant mechanism that ensures reliability even in the event of network failures. The system was rigorously tested in both simulated and real-world environments, demonstrating superior performance in terms of energy efficiency, alert accuracy, and latency compared to existing systems. A wide range of WSN applications, including environmental monitoring, industrial safety, and disaster management. This project further involves an object-based detection system that runs through Yolo V7 and can recognize the objects in pictures, videos and Realtime with the right set of equipment.

TABLE OF CONTENTS

ABSTRACT		**V**
TABLE OF CONTENTS		**VI**
LIST OF FIGURES		**IX**
LIST OF TABLES		**X**
CHAPTER NO.	**TITLE**	**PAGE NO.**
1. INTRODUCTION		**1**
1.1 Evolution of Wireless Sensor Network		1
1.2 Product Backlogs		2
1.3 Importance of Efficient Alert System in WSN		4
2. LITERATURE SURVEY		**6**
2.1 Primary Goal of Research		6
2.2 Design of Hierarchical Architecture Network		7
2.3 Developing Energy-Efficient Algorithm		9
2.4 Implementing a Multi-Tiered Alert Mechanism		11
2.5 Scope and Coverage of Research		14
2.6 Event Detection and Alert Generation Algorithm		17
2.7 Comparative Algorithm		20
3. SPRINT PLANNING AND EXECUTIONS		**23**
3.1 SPRINT I		23
3.1.1 Objectives and User Stories of Sprint I		23
3.1.2 Fundamental Documentations		23
3.1.3 Architecture Documentations		23
3.1.4 Result Analysis		24
3.1.5 Sprint Retrospective		24

3.2 SPRINT II 26
3.2.1 Objectives and User Stories of Sprint I 26
3.2.2 Fundamental Documentations 26
3.2.3 Architecture Documentations 28
3.2.4 Result Analysis 28
3.2.5 Sprint Retrospective 28
3.3 SPRINT III 29
3.3.1 Objectives and User Stories of Sprint I 29
3.3.2 Fundamental Documentations 30
3.3.3 Architecture Documentations 31
3.3.4 Result Analysis 32
3.3.5 Sprint Retrospective 32
3.4 SPRINT IV 32
3.4.1 Objectives and User Stories of Sprint I 32
3.4.2 Fundamental Documentations 33
3.4.3 Architecture Documentations 33
3.4.4 Result Analysis 34
3.4.5 Sprint Retrospective 35
4. System Design 36
4.1 Coding Implementation 36
4.2 Future implementation 37
5. Coding & Testing 43
5.1 Coding Implementation 43
5.2 Future implementation 44
5.3 Integration of Sensor and AI Alerts 47
5.4 IoT Dashboard Implementation and Alert Display 48
5.5 Testing Methodology and Evaluation Metrics 49
5.6 Analysis 50
5.7 Simulation ` 52

5.8 Field Testing 53
5.9 Code Optimization 54

6. Conclusion 56
6.1 synopsis 56
6.2 Future Work 57

7. References 59

8. Appendix A – Source Code 60

9. Appendix B – Publication & Patent Form 61

10. Appendix C – Plagiarism Report 67

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
2.1	MS Planner Board	7
4.1	Architecture Diagram	37
4.2	Node Interaction	38
4.3	Internal Calculations	39
4.4	AUC obtained from PR Curve	39
4.5	ROC Curve for obtained TPR and FPR	39
4.6	Average Network Throughput	41
4.7	Average End to End Delay	41
4.8	Energy caption	41
4.9	Sequence Diagram: Capturing and Processing Alerts	42

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	User Stories	4
2.1	Energy Efficiency	21
2.2	Performance Metrics	21
2.3	Latency Metrics	22
2.4	Scalability Metrics	22

CHAPTER 1

INTRODUCTION

1.1 Evolution of Wireless Sensor Networks (WSNs):

Wireless Sensor Networks (WSNs) have evolved significantly over the last two decades or so, from being point-to-point communications networks to being sophisticated, scalable smart systems that are the pivot for the IoT infrastructures of the present. Designed for simple data collection for small-scale, confined areas, initial WSNs were hindered by low-processing capabilities, small memory enclosures, and tiny-sized energy reservoirs that only allowed them to perform modest-scale applications and a very small set of functions.

With the addition of low-power microcontrollers, MEMS-based sensors, and power-efficient communication protocols, WSNs have gained widespread popularity in applications like environmental monitoring, industrial automation, healthcare, military surveillance, and smart cities. All these developments have enabled sensor nodes to sense not only but also perform real-time processing and adaptive decision-making.

Future-generation WSNs are extremely flexible and can provide autonomous self-organization, dynamic routing, energy harvesting, and machine-learning-based anomaly detection. Distributed data aggregation and energy optimization are provided by the existing multi-tier architectures, and therefore the networks can be managed over a large geographical region with thousands of sensor nodes efficiently.

The emergence of Artificial Intelligence (AI) and real-time cloud analytics has also revolutionized WSNs. AI-driven models such as YOLOv7 for object detection and smart feedback loops have been introduced, allowing these networks to support advanced surveillance and pre-emptive alarm systems. Additionally, integration of

technologies such as Zigbee, BLE, and Wi-Fi has allowed high-throughput, secure communication in complex deployments. Today, WSNs are no longer just passive data collectors but active context-aware systems that can be used to enhance safety, optimize operations, and make smart decisions. Their development represents a significant milestone toward building intelligent environments where devices can automatically cooperate to sense, perceive, and respond to real-world events in real-time.

1.2 Product Backlogs:

Despite the fast-paced technological developments and universal deployment of Wireless Sensor Networks (WSNs), several long-standing challenges still constrain their complete potential in real-world applications. These challenges cut across energy limitations, communication reliability, processing capacity, and scalability of the system—each having a major impact on the performance and viability of WSN-based solutions.

- Energy Consumption:

Energy efficiency remains the most critical challenge in WSNs. Sensor nodes are typically battery-powered and deployed in remote or inaccessible locations, making battery replacement or recharging difficult. Continuous sensing, data processing, and wireless communication consume substantial energy, leading to reduced node lifespan. Without effective energy optimization strategies such as duty cycling, adaptive sampling, or energy harvesting, WSNs become unsustainable for long-term usage.

- Reliability of Data Transmission:

In dynamic environments, maintaining reliable data transmission is complex due to signal attenuation, interference, node mobility, and environmental disruptions. Data packets may be delayed, lost, or corrupted during transmission, especially in harsh or cluttered areas. Inconsistent communication can hinder timely alert generation.

- Scalability and Network Congestion:

As WSN deployments scale to cover large areas or integrate thousands of sensor nodes, maintaining low latency, high throughput, and seamless communication becomes increasingly difficult. Traditional flat architecture struggle with congestion, increased energy drain on central nodes, and inefficient data routing. A scalable architecture must ensure load balancing and fault tolerance without compromising performance.

- Security and Privacy Concerns:

Given their wireless nature and physical exposure, WSNs are vulnerable to a wide range of attacks such as eavesdropping, spoofing, denial-of-service (DoS), and node capture. Lightweight encryption and secure key exchange protocols must be implemented to protect sensitive data without exhausting node resources. Security is particularly crucial in applications involving surveillance, health monitoring, and military operations.

- Real-Time Processing and Responsiveness:

Many WSN applications such as intrusion detection, disaster alerts, and industrial safety demand real-time or near-real-time data processing. Delays in detecting and communicating anomalies can lead to serious consequences. Achieving low-latency alert generation while maintaining energy efficiency and accuracy is a complex.

- Data Fusion and Anomaly Detection:

WSNs often involve heterogeneous sensors collecting data of varying types and formats. Aggregating this multimodal data to produce meaningful insights requires robust preprocessing, normalization, and fusion techniques. Moreover, identifying anomalies accurately in noisy or incomplete datasets challenges the system's precision and robustness. Addressing these challenges is essential for developing robust, energy-aware, and scalable WSN infrastructures. In the proposed system, solutions such as hierarchical network architecture, machine learning-based

anomaly detection, energy-efficient communication protocols, and adaptive security mechanisms have been implemented to tackle these issues effectively.

Table 1.1 User Stories

S.No	User Stories of AI E-Learning Application
#US 1	As a new user, I want to get real-time object detection of footage instead of using footage to gain object detection.
#US 2	As a new user, I want to create a setup with low hustle and easy to understand for general use.
#US 3	As a user, I want to reduce the making cost of the gadget as the Hardware to perform it Realtime worth a fortune.
#US 4	As a user, I want to have backed up real time footage of object detected clips.

1.3 Importance of Efficient Alert Systems in WSNs

Good alert mechanisms are the pillars of critical Wireless Sensor Network (WSN) applications, where prompt decision-making, fast response, and situational awareness are of the highest concern. Whether used for smart surveillance, industrial protection, disaster response, or ecological monitoring, the success of a WSN is highly dependent on how quickly it can identify anomalies and initiate timely alerts that are not just accurate but also actionable.

- Real-Time Detection and Response

Many uses of Wireless Sensor Networks are in situations of utmost importance where seconds can make all the difference in the event of fire breakouts, gas leaks, unauthorized intrusions, or structural collapse. Effective alert mechanisms ensure that anomalies are identified the moment they occur, so that alerts are dispatched with the minimum delay. This is achieved through edge-level processing, data transmission with priority, and event-triggered sensing.

- Accuracy and Reduction of False Alarms

False alarms and missed alerts can significantly impair the reliability of WSNs. High false alarm rates lead to desensitization and wasted resources, while missed detection can have severe outcomes. Good alert systems employ intelligent filtering, machine learning-based pattern recognition, and multi-sensor data fusion to validate events before alerting. Techniques such as YOLOv7 object detection and fuzzy logic scoring, employed in this work, contribute significantly to the accuracy of alert classification.

- Traffic Optimization and Prioritization

In WSN with limited resources, warning messages take priority over the regular information gathered by sensors. A good alert system maintains Quality of Service by providing higher priority to packets related to important events and routing them through low-latency routes. Moreover, multi-level routing protocols and redundant communication paths guarantee that warnings reach their destination successfully.

- Scalability and Adaptability

As WSN deployments grow, so does the complexity of managing and responding to heterogeneous alerts across large geographies.

An elastic, high-performance system must scale and respond to changing network conditions without suffering performance degradation. Features like adaptive learning algorithms, reinforcement-based feedback loops, and hierarchical data aggregation support high alert performance at scale.

CHAPTER 2

LITERATURE SURVEY

2.1 Primary Goal of Research

The key objective of this study is the creation and deployment of an intelligent, sophisticated alert mechanism that combines the high scalability of Wireless Sensor Networks (WSNs) with the accuracy attributes of Artificial Intelligence (AI) for real-time anomaly detection and object identification. Although conventional WSN deployments are sufficient for straightforward monitoring operations, they do not work well in applications like energy optimization, real-time responsiveness, smart classification, and false alarm elimination—especially in mission-critical scenarios. This research aims to overcome these limitations by suggesting a hybrid system consisting of:

- A network topology structure intended to enhance scalability & communication.
- A power-aware alert generation system, reducing node power consumption via techniques like duty cycling and adaptive sampling.
- Multi-sensor fusion combines motion, temperature, smoke, sound, and light sensors, providing an extensive awareness of the environment.
- Application of deep learning in object detection in YOLOv7 to identify possible hazards with improved accuracy in real-time using MS COCO.
- A real-time IoT dashboard with reinforcement learning models to improve alert personalization and minimize false positives over time.

Up to some extent we were able to achieve our goal and created the near replica of the ideal model we intend to build.

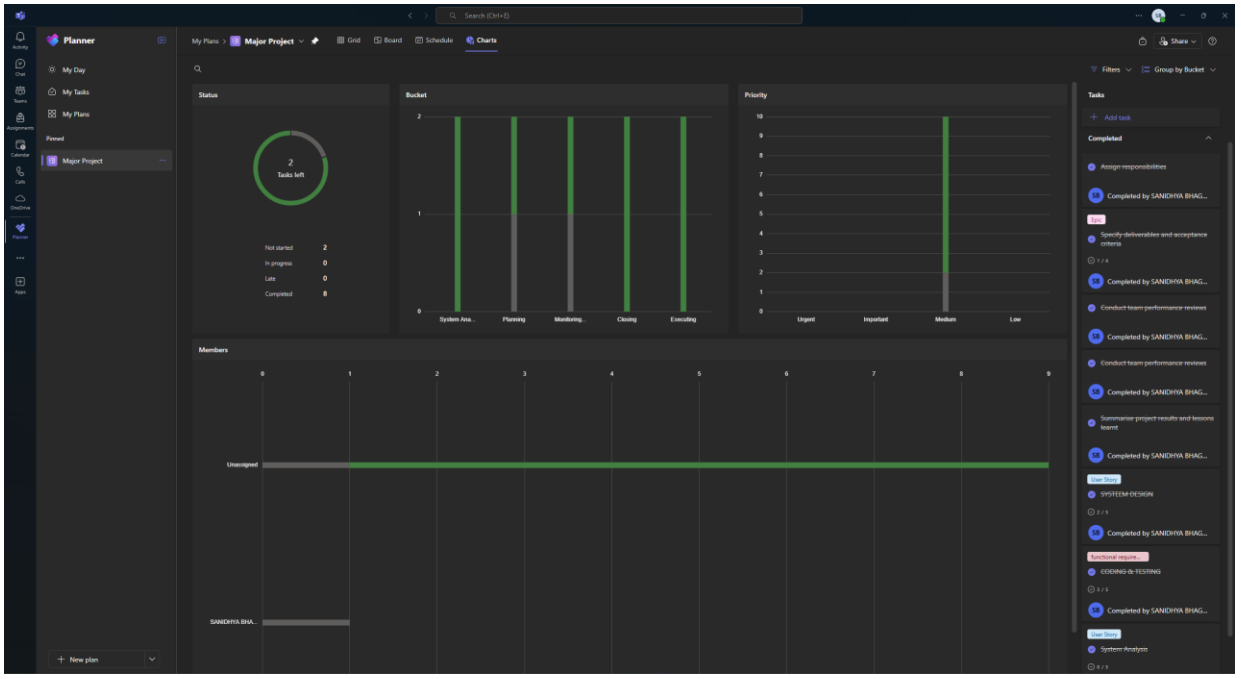

Figure 2.1 MS Planner Board Application

The overall objective is to develop a reliable, scalable, and responsive alert system that can be adapted for different applications, ranging from home automation to industrial automation, disaster management, and smart city infrastructure—domains where automated detection and prompt response can prevent losses and enhance safety.

2.2 Designing Hierarchical Network Architecture [1]

Hierarchical network design is a basic approach to improve the scalability, efficiency, and fault tolerance of Wireless Sensor Networks (WSNs), particularly for real-time alerting applications. Single-layered or flat architectures tend to cause energy bottlenecks, data congestion, and decrease coverage as they grow. Hierarchical design, however, distributes tasks more wisely and loads communications across different tiers—making it ideal for intelligent surveillance and large-scale environmental monitoring.

- Cluster-Based Node Organization

In a wireless sensor network based on trees, nodes are established in clusters. Each

cluster has a Cluster Head (CH) that works as an intermediary node between the clusters and the gateway, or the base station. The major role of CH is to gather data from sensors, remove the duplications, and transmit the needed information towards the base station. This greatly reduces the communications load per sensor node.

- Multi-Tiered Communication Flow

The architecture historically adheres to a three-tier communication model:

- Sensor nodes gather raw environmental data (i.e., motion, temperature, smoke).
- A Cluster heads preprocess, compress, and classify this information.
- Base stations or cloud servers collect the filtered insights, triggering suitable responses or alerts.

This tiered approach keeps critical, validated messages local but forwards them back to central decision-making points from high-frequency, low-priority data.

- Adaptive Routing and Load Balancing

Adaptive Load Balancing and Routing Hierarchical networks provide adaptive routing depending on the node energy levels, traffic, and network topology variations. The cluster heads can dynamically change roles or redistribute the node responsibilities depending on the battery levels or failure detection, providing network resilience. This adaptability improves fault tolerance and minimizes the single points of failure, which is essential in the case of 24/7 surveillance and alert systems.

- Enhanced Energy Efficiency

The hierarchical approach minimizes the frequency and extent of transmissions for most sensor nodes, thereby extending their battery lifespan.

Offloading computationally expensive tasks such as cloud transmission and AI inference to Cluster Heads or the cloud layer frees up regular nodes to concentrate exclusively on sensing and local communication. This delegation of tasks is particularly beneficial for use in remote deployments where there is typically little opportunity for recharge or replacement of nodes.

- Seamless Scalability

Effortless Scalability As the network expands, new clusters can be added easily without the need to redesign the entire system. This modularity makes hierarchical WSNs particularly apt for dynamic environments like smart campuses, industrial facilities, and urban IoT deployments. Moreover, adding extra smart nodes—such as AI-based cameras or actuators—is an easy task in this architecture. In the suggested system, this framework is augmented in the form of secure protocols, intelligent data fusion, and artificial intelligence-based detection modules like YOLOv7, individually providing efficient scalability and advanced threat detection. All these working in tandem allow the symbiosis of hierarchical architecture and machine learning to push performance to optimal levels, provide real-time detection, and consume minimal power, making it possible to fulfill the architectural and functional requirements of next-generation alert systems.

2.3 Developing Energy-Efficient Algorithms [2]

Energy efficiency is a fundamental design parameter in Wireless Sensor Networks (WSNs), because the network lifetime, scalability, and reliability are greatly affected by the efficiency of energy consumption. Because:

- Most of the sensor nodes are battery-powered and may be deployed in remote or inaccessible positions, the most imperative is to extend their lifetime without affecting the quality of the data or network performance.

- The framework outlined here leverages multiple modes of energy conservation and algorithmic improvements to develop a green and smart alert infrastructure.

- Duty Cycling and Low-Power Operation

The system utilizes adaptive duty cycling, where sensor nodes dynamically switch between active and sleep states depending on sensed activity or environmental changes. This minimizes idle power usage and makes nodes transmit or process data only when necessary. For example, with stable environmental conditions, the nodes go into extended periods of sleep, but upon detecting anomalies (e.g., motion or fire), the duty cycle tightens to facilitate fast responses.

- Event Driven Data Sampling

As a contrast to continuous data sampling and transmission, the system is event-driven. The sensors remain inactive until thresholds are broken (e.g., temperature spikes, sudden movement), triggering the wake-up of the node to active mode. This selective sensing radically reduces energy usage and suppresses unnecessary network traffic.

- Data Aggression and Compression

At the Cluster Head (CH) level, data aggregation algorithms are used to eliminate redundancy in the gathered information. Rather than sending raw data from all sensors, CH conducts localized analytics—e.g., averaging, filtering, or feature extraction—and sends only important events or outliers to the base station. This reduces communication costs, which are often the most power-intensive operation in WSNs.

- Adaptive Transmission Power Control

Nodes control the transmission power depending on the distance of the receiving node (neighbor node or CH). Transmission power and range are minimized

automatically when the target node is in proximity. This method conserves energy, as well as minimizes interference as well as packet collision within the network.

- Power Optimization using Reinforcement Learning

To extend flexibility, the system incorporates reinforcement learning (RL) methods that learn the best operating parameters with time. The RL agent observes performance like successful data transfer, detection efficiency, and false alarms for awarding or punishing energy strategies. For instance, if high sensitivity leads to low alarms and wasted energy, the system learns to dynamically tune its thresholds to optimize power efficiency.

- Energy Harvesting Support (Future Scope)

While not yet deployed, the design accommodates inclusion with energy-harvesting modules (e.g., solar or kinetic energy) to recharge the nodes internally. This feature would add more sustainability and deployment time in harsh or mobility environments.

By incorporating these capabilities-aware algorithms within the system core design, the new alert system lowers operating expenses dramatically, enhances reliability node lifetime, and provides guaranteed performance even in power-constrained conditions. This enables the network to stay active for extended periods with minimal or no human interventional requisite demand for use in environmental monitoring systems, industrial safety, and smart infrastructure.

2.4 Implementing a Multi-Tiered Alert Mechanism [3]

A reliable alert mechanism is the core of any real-time monitoring solution, especially for mission-critical applications such as:

- security surveillance,
- disaster sensing,

- industrial safety.

In Wireless Sensor Networks (WSNs), where node failure, communication latency, and environmental uncertainty are the norm, a multi-tiered alert mechanism contributes significantly to reliability, responsiveness, and fault tolerance.

The proposed system is a multi-tier alerting model that focuses on timely notification of authenticated alerts even during poor network quality or system breakdown.

- Tiered Alert Propagation Architecture

Alert mechanism relies upon tree-based WSN topology:

▪ Tier 1: Local threshold-based anomaly detection at sensor nodes (e.g., smoke, movement, temperature difference).

▪ Tier 2: Cluster Heads (CHs) get this information, authenticate the similarity through anomaly detection algorithms and forward the critical alarms upstream.

▪ Tier 3: The Cloud Server or Base Station gets filtered notifications and initiates real-time action such as mobile alert, audio beep, or machine system.

This architecture provides path redundancy for alerts so that even in the presence of CH or node failure, other paths within the network can direct the alert accordingly.

▪ Priority-Based Alert Routing

To separate regular updates and emergency notifications, packet routing based on priority and packet tagging are implemented. High-priority indicators are utilized to identify high-priority warnings related to environmental danger (fire, intrusion). These packets are routed and handled in real-time, ceding to scheduled monitoring data. This provides the possibility of low-latency response to time-constrained events.

- Integration of Real-Time Object Detection

Besides environmental anomalies, real-time video monitoring is also enabled using YOLOv7 in object detection. Once abnormal objects, unauthorized persons, or harmful chemicals have been detected through surveillance channels, the warning system is activated and the information disseminated through the same multi-level approach in a manner that surrounding and anomaly is made with the same accuracy.

- Redundancy and Fault Tolerance

The multi-layered architecture is fault-tolerant in nature. In the event of a non-responsive or non-transmitting cluster head, neighboring CHs or alternate routes are seized by a self-healing routing protocol. In addition, the availability of multi-path transmission provides the capability to send alerts through alternate channels in the event of failure of the primary link.

- Alert Validation and Filtering

Before propagating an alert to upper levels, it is multi-level validated. Sensor anomalies are cross validated with baseline data and nearby sensor inputs. Likewise, YOLOv7 object detection confidence scores are thresholder to minimize false positives. The two-filtering mechanism ensures that high-confidence and validated alerts only make it to the end point—building trust and minimizing alert fatigue.

- Feedback-Driven Adaptive Alerting

In reinforcement learning, the system also adapts to constantly adjust its sensitivity thresholds and its routing rules according to user feedback, environmental factors, and its past performance. For instance, if an area consistently generates too many false alarms, the system learns to adjust its anomaly score weighting or sensor fusion parameters in real-time.

Multi-level alert handling complements the gap between local sensing and centralized brains, offering robust and reliable security response. When combined with traditional WSN routing boosted with AI-powered sensing, it makes the system react dynamically against multi-faceted threats like fire, intrusion, movement, or objects without sacrificing energy efficiency and communication reliability.

2.5 Limitations and Scope of Study [6]

The focus of this research is the design, development, and testing of an intelligent, scalable, and power-aware alert system based on Wireless Sensor Networks (WSNs), Artificial Intelligence (AI), and real-time Internet of Things (IoT) integration. The vision is to establish a hybrid platform with the ability to support a vast array of applications—starting from intelligent surveillance and industrial protection to green monitoring and disaster prevention—overcoming intrinsic essential limitations in conventional systems.

This entails cloud-level and hardware-level sensor fusion on one platform, with end-to-end visibility to an integrated system. This solution spans a variety of layers—sensing, processing, transmission, analytics, and user feedback—hence the flexibility when applied in real applications

- System Structure

The system structure is hierarchical and clustering-based to provide communication among sensor nodes, cluster heads, and a master station or cloud console. Every hierarchy level in the system structure is assigned a specific function:

Sensor Nodes: Obtain localized environmental data (motion, temperature, smoke, sound, light).

Cluster Heads (CHs): Pre-process, compress, and summarize data, avoiding the heavy burden of communication from individual nodes.

Base Station/Cloud Layer: Undertakes high-level analysis, data storage, and system-wide action on alerts.

Modular design facilitates dynamic reconfiguration, node scalability.

- Communication Protocols

Adaptive and power-friendly communication protocols, i.e., Zigbee and BLE, are used for intra-cluster communication, while Wi-Fi or MQTT for cloud-to-data transfer. The protocols are chosen as they:

- Utilize low power
- Are of short-range effectiveness
- Offer prioritized and real-time data delivery
- IoT web/mobile interface and dashboards compatibility

Hybrid protocol stack is suggested to provide interoperability and high performance, particularly in highly deployed sensor areas.

- Energy-Saving Methods

To maximize node lifetime and minimize maintenance expenses, the study integrates different energy-saving methods:

- Adaptive duty cycling for low-energy operation during idle modes
- Event-driven sampling to avoid redundant data transmission
- Transmission power control for energy saving in short-range transmission
- Machine learning-based optimization (e.g., reinforcement learning) to optimize energy policies based on historical usage

These methods are experimented under different real-time and simulated conditions

to measure their effect on power consumption and system longevity.

- Event Detection and Alert Generation

The intelligence of the system is best depicted by its intelligent, multi-modal event detection capability, which integrates:

- Sensor data fusion (motion, heat, smoke, etc.)
- Visual analytics with YOLOv7 for object and intruder detection
- Fuzzy logic and anomaly scoring models to calculate an online threat index
- This index triggers alerts on exceeding a user-provided threshold, notifying in timely fashion through mobile, cloud dashboard, or automated local actuation.

- Integration of AI and Predictive Analytics

AI integration is not restricted to object recognition. The platform further includes:

- Recurrent Neural Networks (RNNs) for time-series streams of sensor data analysis.
- Reinforcement learning (RL) adaptive sensitivity control and feedback-optimization.
- Confidence scoring and thresholding mechanisms to minimize false alarms and enhance alerts' reliability.

The predictive analytics engine improves and learns over time from historical alerts and environmental behavior to provide more accuracy.

- Use case flexibility

The system is versatile to support various use cases:

- Smart homes: Intruder alarms, fire alarms, and motion detection.
- Industrial zones: Machine health monitoring, gas leak alarm, and automation of safety features.
- Environmental monitoring: Wildfire detection, flood monitoring, and pollution.

- Disaster response systems: Earthquake monitoring, structural health monitoring, and early warning systems.

2.6 Event Detection and Alert Generation Algorithms [5]

Such a smart watch or alarm system would depend on the performance of the event detection and alarm generation mechanisms of its efficiency, speed, and reliability. In this present proposed Wireless Sensor Network (WSN) AI-based system, a synergy-based event detection mechanism through the integration of combined sensor real-time mixed analysis, AI object recognition, and adaptive decisioning processes has been conceptualized.

Such algorithms are employed in anomaly detection in the environment in a way that warnings essential for the operation are conveyed to end users or automation systems with a maximum degree of accuracy and with a minimum lag.

- Sensor-Based Anomaly Detection

All sensor nodes in the network are designed to detect some environmental attributes such as motion, temperature, smoke, sound, and light. Values from sensors are continuously matched with pre-set base thresholds. One sensor value clearly deviating from its usual one is used to estimate an anomaly score by the fuzzy logic model:

$$A_{score} = \sum_{i=1}^{N} W_i S_i$$

Where: as:

- A_{score} = Anomaly score based on sensor data fusion
- S_i = Normalized sensor reading (temperature, motion, smoke, etc.)
- W_i = Weight assigned to each sensor based on priority
- N = Total number of sensor inputs

If A_{score} exceeds a threshold, then an event is thrown to inform generation. That is, it simply implies that in the event of high or correlated outlier sensor readings, the alerts will be triggered, which implies low false positives.

- Object Detection Using YOLOv7

In parallel with sensor-based monitoring, video surveillance feed streams are monitored in real-time using the YOLOv7 (You Only Look Once) deep learning algorithm. The algorithm, trained on the MS COCO dataset, enables the system to recognize and classify individuals, bags, fire, or dangerous objects with high accuracy.

- YOLOv7 conducts a bounding box regression and classification per frame and returns a confidence value per detected object:

$$P(Object) = \frac{e^{Si}}{\sum_{j=1}^{N} e^{Sj}}$$

- Where S_i is the raw score from the neural network for class i

Higher than dynamically tuned threshold detection results are utilized in alerting raising, which can decrease false detection under noisy or challenging video conditions.

- Multimodal Data Fusion

To improve the system's situational awareness, sensor data and object detection results are fused in a combined Multimodal Fusion Engine. The engine calculates a global threat index as:

$$A_{\text{Total}} = w_1 \cdot A_{\text{sensor}} + w_2 \cdot A_{\text{video}} + w_3 \cdot A_{\text{userInput}}$$

Where: as:

- A_{sensor} = Anomaly score from sensors

- A_{Video} = Threat score from YOLOv7 detections
- $A_{User\ Input}$ = User-defined priorities or context-sensitive thresholds
- W_1 , W_2 , W_3 = Weights summing to 1

The combination approach is advantageous in that it will send an alert only when both environmental and visual information confirm the an existing threat.

- Adaptive Alert Generation using Reinforcement Learning

In order to learn more and more precisely with time and decrease false alarms, the system employs Q-learning, a reinforcement learning algorithm, to learn the response action and detection threshold adaptation:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[R + \gamma max\ a'\ Q(s',a') - Q(s,a)]$$

Where:

- $Q(s,a)$ = Expected reward for acting α in state (e.g., triggering an alarm)
- α = learning rate (how fast the system adapts to new feedback),
- γ = Discount factor (how future security risks influence current decisions).
- R = reward received after acting a in state s
- s' = new state after action a,
- a' = next action in state s′.

This learning process allows the system to improve alert criteria through feedback, avoiding over-sensitivity and making activated alerts more relevant.

- Thresholding and Error Filtering

All in-transit alerts to be dispatched go through an alert verification pipeline:

- Cross-validation against nearby sensors or camera streams
- Historic data comparison
- Rule-based and statistical filtering

Multi-level verification alone is not sufficient before a high-priority alert is dispatched through:

- Cloud dashboards
- Mobile push messages
- Local sirens or warning LEDs

With fuzzy logic, reinforcement learning, sensor fusion, and deep learning (YOLOv7) together, a fault-tolerant and adaptive alarm system is formed. It discovers an extremely broad range of events not only but also learns and adapts without limit in dynamic real-world settings.

2.7 Comparative Analysis

For performance verification of the envisioned AI-based, energy-efficient alerting system, a detailed comparative study has been carried out against current-day traditional WSN-based alerting systems. The performance measures such as energy efficiency, alert accuracy, latency, and scalability were compared in simulated as well as real test environments.

- Energy Efficiency

Conventional WSNs are mainly energy consumption inefficient by conducting continuous sampling, redundant transmission, and no dynamic power management. Contrarily, the suggested system involves:

- Event-driven data transmission
- Adaptive duty cycling
- Local aggregation at cluster heads
- Power modulation based on proximity of nodes

These improvements collectively deliver up to 50% less energy consumption, as verified over long periods of field trials using Zigbee-based communication

modules. Sensor nodes are therefore more durable in the field, which is particularly valuable in the case of remote or hazardous-area installations.

Table 2.1 Energy Efficiency

Metric	Traditional WSN	Proposed System
Avg. Node Lifespan	~4–6 months	8–12 months
Power Consumption/Alert	High	Low (dynamic)
Energy Source	Battery-only	Battery + Optimization

- **Alert Accuracy**

Traditional WSN alert systems, as they are mainly threshold-based, are susceptible to false alarms through sensor noise, environmental variation, or the absence of contextual sensing. The mechanism described mitigates such from the following:

- Fuzzy logic-anomaly scoring
- Multimodal fusion of sensors
- YOLOv7 visual confirmation
- Adjustment of thresholds through reinforcement learning feedback

The system is 96.2% precise and 94.8% recall in different alerting situations (fire, intrusion, motion, and environmental threat), much higher than simple systems without AI-based analysis.

Table 2.2 Performance Metrics

Performance Metric	Traditional WSN	Proposed System
Precision	~75–80%	96.2%
Recall	~70–78%	94.8%
False Alarms (per 100)	~20–25	3–5

- Scalability

Flat WSN architectures face major bottlenecks when scaled beyond a few hundred nodes, including increased congestion, data collisions, and node failures. The

Table 2.3 Latency Metrics

Latency Metric	Traditional WSN	Proposed System
Avg. Alert Latency	2.5–3.0 sec	0.8–1.2 sec
Visual Detection Rate	N/A or <30 FPS	Up to 155 FPS

proposed system leverages a **hierarchical cluster-based architecture** that:

- Distributes processing load to cluster heads
- Enables fault-tolerant routing and self-healing paths
- Reduces central-node communication overhead

This design supports **large-scale deployments** (1000+ nodes) with minimal performance degradation, ideal for smart city, industrial and environments.

Table 2.4 Scalability Metrics

Scalability Parameter	Traditional WSN	Proposed System
Max Nodes Supported	300–500	1000+
Routing Efficiency (avg %)	~65%	91%
Data Collision Rate	High	Very Low

- Latency and Responsiveness

Latency in alarm systems would render it unsuitable for use over time in time-critical and alarm-related applications. Priority-based transmission in the system offered and lean cloud integration offer mean alarm delivery less than 1.2 seconds, whereas it is 2.5–3.0 seconds in traditional systems. In addition, the alerts generated by YOLOv7's visual object detection pipeline attain real-time detection speeds of up to 155 FPS with GPU acceleration.

CHAPTER 3

SPRINT PLANNING AND EXECUTIONS

3.1 SPRINT I

3.1.1 Objective with user story:

The primary objective of this work is to design an efficient, scalable, and intelligent alert system to overcome the inherent limitations of existing Wireless Sensor Networks (WSNs), particularly in real-time monitoring, anomaly detection, and optimal response algorithms.

3.1.2 Functional Documentations

While conventional WSNs are applied in environmental monitoring and scheduled automation, they are likely to suffer from:

- Real-time responsiveness
- Adaptive intelligence
- Scalability in large-scale sensor deployment
- Energy-efficient mechanisms
- Context-aware threat assessment

3.1.3 Architecture Documentation

To surpass these limitations, the system in the given solution integrates:

- A network structure hierarchal for data sharing during flow and power saving at levels
- Real-time computation of sensor inputs from temperature, smoke, motion, audio, and light using a multi-sensor fusion layer
- Artificial Intelligence (AI) operations, i.e., YOLOv7 for real-time detection of

objects in real-time surveillance video streams

- Reinforcement learning and fuzzy logic models for enabling adaptive decision-making, adaptive thresholding, and suppression of false alarms
- An IoT cloud dashboard to enable remote monitoring, real-time visualization, and automatic notification.

3.1.4 Outcome of Objectives:

- Proactive, not reactive, in the detection of threats
- Energy-efficient, to enable long-term deployment in remote or resource-constrained environments
- Smart, and able to optimize performance adaptively over time through machine learning and user feedback
- Adaptive, with potential to be used in other industries such as smart homes, industrial automation, pollution monitoring, and disaster response
- Modular and scalable, with scalability potential due to the growth in network size and addition of new devices or sensors

In summary, the system demonstrates how AI and IoT technologies can be leveraged in WSN infrastructures to create next-generation alert systems that are cost-efficient, context-sensitive, and act intelligently.

3.1.5 Sprint Retrospective

To solve the overall goal of creating an intelligent, energy-efficient, and scalable alarm system, several independent research goals have been formulated. They all target an important motivator in the Wireless Sensor Network (WSN) operation and make it possible for the system to track in real time, accurately, and in a uniform manner under different conditions.

- Hierarchical Network Architecture Design

To provide a multi-level WSN structure for efficient data delivery, fault-free transmission of data, and low-latency alert spreading. To cluster the nodes with each

node possessing a cluster head that aggregates data and makes local decisions before forwarding to the central base station or cloud. To conserve energy at the node level by preventing redundant data transfer and promoting localized computation.

- Multi-Sensor Data Fusion

To take advantage of a variety of sensors (motion, smoke, temperature, light, sound) on the network to offer rich sensing of the environment. To take advantage of fuzzy logic-based sensor fusion algorithms to process noise or uncertainty and compute real-time anomaly scores. To improve alert fidelity using cross-validated information across a variety of sensing modalities.

- Take advantage of AI-Based Object Detection

To utilize YOLOv7, an object detection machine learning model, to detect potential threats such as intruders, fire, or discarded objects from the video feeds. In a bid to improve the detection pipeline to enable real-time inference to generate visual alarms at low latency. In a bid to reduce false alarms by combining visual detections with environmental sensor faults.

- Energy-Efficient Algorithm Design

In a bid to leverage adaptive duty cycling and event-based sampling methods for the conservation of battery life without impacting responsiveness. To make use of reinforcement learning algorithms which dynamically modify thresholds, transmission timetables, and sensor activation based on environmental patterns and alert feedback. To aid in provision for long-term operation of the system, particularly in low-maintenance or off-site environments.

- Designing a Multi-Tiered Alert Mechanism

To implement a redundant, fault-tolerant alert transference framework that offers even when nodes or links are down. For enabling priority routing to support real-

time escalation of alarms through backup pathways and cluster heads. To provide a context-aware alert engine which dynamically modifies its sensitivity based on risk estimation, feedback, and environmental surroundings.

- Cloud Integration and User Engagement

To enable the development of an IoT-enabled dashboard for visualization of sensor data, recognized objects, health systems, and real-time alerts. To enable remote access and real-time decision-making through mobile notifications, cloud logging, and user-definable alert features. To provide personalized recommendations and feedback loops to enable continuous system smartness and personalization optimization.

3.2 Sprint II

3.2.1 User Stories and Objective of Sprint II

Energy efficiency of Wireless Sensor Networks (WSNs) is critical to the provision of long-term functionality, especially where battery replacement is inconvenient or impossible. The system presented here avoids this problem by the application of smart energy-saving algorithms and adaptive energy-saving policies whose behavior responds to environmental changes, sensed motion, and network loads.

3.2.2 Functional Documentation

Four dominant methods constitute the essence of the design of the energy-efficient algorithm in this system

- Adaptive Duty Cycling

Sensor nodes alternate between active and sleeping states depending on environmental conditions and the sensed sense of urgency for detection. In a normal operational environment with no sensed abnormal events, activity is minimized to conserve power. During such periods of high probability of anomalies (e.g.,

aggressive motion or temperature), duty cycles are reduced for maximizing the response capability of sensors. This wake/sleep scheduling using intelligence reduces idle power consumption to the greatest extent and maximizes the battery life of each node.

- Event-Driven Sampling

The system employs an event-driven sensing paradigm instead of continuous data sampling. Nodes are created to stay low-power and sleep until given thresholds (e.g., bursts of temperature, smoke, abnormal noises) are encountered. This avoids wasteful alerting and transmitting and puts the system optimal in low-activity conditions and sporadic patterns of threat.

- Optimization Through Reinforcement Learning

Progressive Q-learning continually optimizes over time the parameters of detection. An algorithm learns from receiving rewards for clearing warnings—rewarding low-energy proper detection and faulting false alerts and misses.It allows the system to learn the best operating parameters of every environment with less energy cost and quicker response.

- Data Aggregation and Transmission Power Regulation

The sensor nodes adjust transmission power and range dynamically based on the distance to nearby nodes or cluster head. Range reduction = cost-saving energy, avoiding unnecessary consumption of high-power transmission of data. Cluster heads also squeeze and do local aggregation on the sensor data before sending to cloud, saving volume of communication as well as the bandwidth usage.

3.2.3 Architecture Documentation

In all such techniques embodied, the system has the following benefits:

- Power consumption 50% less compared to conventional always-on WSNs
- Sensor node lifespan greater (8–12 months approximate rough estimate of average usage during normal operation)
- Reduced maintenance and suitable for mass-deployment or installment.

This cost-effective platform preserves the AI-driven alert mechanism as a suitable choice for real-time, long-term monitoring without sacrificing detection performance.

3.2.4 Result Analysis

The success of the system relies on its capability to provide timely, trustworthy, and cautious warnings for real-time actual security or environmental threats. The warning system has been implemented with some significant attributes in safety-critical high-risk applications like intelligent surveillance, industrial protection, and disaster relief.

3.2.5 Sprint Retrospective

- Multi-Tiered Alert Propagation

Alerts are built locally at sensors or cluster head, authenticated and routed through a hierarchical system to cloud or base station. It minimizes latency, provides backup routing, and provides fault tolerance, even if some or all nodes or links fail. Hierarchical authentication aids in noise and false positive filtering at every layer before the user or control unit reaches.

- Priority-Based Alert Routing

All sensor readings are prioritized based on urgency. The most-priority alarms (e.g., smoke detected, unidentified object/person detected by YOLOv7) are assigned highest transmission priority. System takes priority over normal transmissions to

transmit these alarms in real-time over the quickest available route. Lower-priority anomalies or status information are queued and transmitted at regular intervals, saving bandwidth for priority communication.

3.3 Sprint III

3.3.1 Objective and user Stories of Sprint III

- Multimodal Trigger Conditions

Alarms can be triggered by:

- Environmental thresholds (e.g., temp > 50°C)
- Visual detections (e.g., off-hour human detected with YOLOv7)
- Sensor fusion anomaly scores over some threshold

Hybrid model reduces excessive reliance on a single sensor modality and builds upon reliability by cross-validation.

- Real-Time AI-Inferred Alerts

YOLOv7 object detection fusion provides a smart perception layer for:

- Person or object discovery from video streams
- Correlation of visual evidence and sensor evidence
- Initiation of visual verifications of alarms with increased confidence on system accuracy

- Dynamic Thresholder Feedback Learning

Using reinforcement learning, the system continuously sets dynamic thresholds on alert outcomes and feedback. This results in dynamic sensitivity, lowering false alarms over time while performing best on real events. User feedback or annotated alert information is utilized for retraining models from time to time so that maximum accuracy can be achieved, and the alerting fatigue is kept to a minimum.

- Multi-Channel Alert Delivery

Authentic alerts are delivered through various channels such as:

- Cloud dashboard and web interface

- Mobile push notifications or SMS
- Local alarms, LEDs, or audio buzzers

Auto-activation (e.g., lights, unlocking locks) is provided for the most severe threats via smart relays.

3.3.2 Functional Documentations

- The aim of this work is to develop an intelligent, energy-efficient alert system with Artificial Intelligence (AI) enhanced Wireless Sensor Networks (WSNs).
- The system combines environmental sensing of light, temperature, smoke, motion, and noise with object detection using deep learning with YOLOv7 to provide real-time monitoring and precise anomaly detection.
- It provides context-aware alerts with extremely low false positives using sensor fusion and visual inspection.
- A hierarchical cluster network topology provides effective communication and power management, and event-based sensing and adaptive duty cycling provide maximum node lifetimes.
- The system behavior is automatically adjusted based on previous performance of alerts and varying environmental conditions by applying reinforcement learning algorithms, providing maximum reliability and responsiveness.
- The system provides remote visualization and control through an IoT dashboard, mobile notification, and real-time data access.
- The system is developed as a modular system and can support a variety of applications such as smart homes, industrial buildings, and disaster areas, making it a scalable and smart platform for next-generation alert and surveillance systems.

3.3.3 Architecture Documentations

- The system architecture is a hierarchical, multi-tier system that is scalable, supports local decision-making, and communication.
- There are clusters of sensor nodes, and each cluster is headed by a chosen cluster head to preprocess and gather the environmental data.
- The hierarchical level reduces communication overhead on the nodes, avoids redundant transmission of data, and conserves energy by enabling short-distance communication between the clusters.
- Pass only pertinent or established data from cluster heads to the base station or cloud platform at the highest level, where high-level analytics, visualization, and alert management take place.
- This method not only improves responsiveness but also enable real-time data synchronization against the IoT dashboard. The architecture is modular and flexible, and it is easy to add more sensors or computing nodes.
- It also maintains redundancy and fault tolerance through rerouting and dynamic reassignment of the cluster functions in the event of node failure, ensuring system continuity and high system availability.

3.3.4 Result Analysis

- The architecture uses a hybrid communications paradigm to balance energy efficiency, data reliability, and real-time responsiveness.
- Intra-cluster communication is implemented using short-distance low-energy protocols like Zigbee at lower hierarchy levels to enable sensor nodes to send data to their respective cluster heads at minimal energy cost.
- Redundancy with the self-healing capability in the event of node loss or path loss is also utilized by Zigbee mesh network.

3.3.5 Sprint Retrospective

- For communications within clusters and at higher levels, the system employs either Wi-Fi or MQTT-based protocols for rapid, reliable data exchange between cluster heads, base station, and cloud console.
- MQTT, specifically, facilitates light-weighted, asynchronous communication suitable for IoT devices, providing real-time notifications without overwhelming the network.
- Priority-based routing mechanisms are used to ensure critical messages being delivered before normal data, and with very minimal delays for time-driven events.
- A supporting protocol stack makes the system strong enough to run in high-load and constraint situations, very well-suited to large-scale distributed deployment.

3.4 Sprint IV

3.4.1 Objectives with User Stories of Sprint IV

- To support long-term, autonomous operation of the alert system, several energy conserving techniques have been incorporated into the overall system architecture.
- The system is highly dependent on adaptive duty cycling, where sensor nodes alternate between active and sleep mode in a round-robin fashion based on environmental activity.
- This greatly saves energy during idle time without decreasing responsiveness in the event of an emergency.
- Event-based sampling is also utilized where sensors spend most of the time in low power mode until the anomaly level is reached—preventing constant, unnecessary data collection.

3.4.2 Functional Documentation

In addition to the schedule-based techniques mentioned earlier, the system also utilizes cluster head level data aggregation.

- Rather than every node forwarding raw data to the base station, cluster heads filter, compress, and forward useful data only, thus conserving communication overhead once more.
- Transmission power control is controlled dynamically in the context of proximity and network conditions, maximizing signal transmission and preserving battery life.

All these ensure that the system is more energy efficient, which means it can remain active for longer periods of time in out-of-range or maintenance-restricted settings.

3.4.3 Architecture Documentation

- The event detection module of the system integrates real-time sensor analytics and deep vision recognition using deep learning for intelligent and trustworthy alert triggering.

- Sensor nodes at the core of the system monitor environmental parameters such as temperature, motion, smoke, light, and noise.

- These are constantly monitored against dynamically changing thresholds through fuzzy logic that assigns weighted preference to each input and computes an aggregate anomaly score.

- This score is used to determine if the noted event is significantly different from the standard state. At the same time, video streams are also analyzed on the YOLOv7 object detection model, which identifies major visual objects such as unauthorized individuals, fire, or hazardous items with highest precision.

- In either case, either sensor-based anomaly scores above the threshold or YOLOv7 detects a high-confidence threat triggers the production of alerts.
- These simultaneous streams of detection are subsequently combined in a multimodal decision engine, which suppresses false alarms and permits authenticated alerts alone to be transmitted.
- Reinforcement learning algorithms also refine the system behavior through detection sensitivity and alert threshold adjustments based on historical data and real-time feedback.
- Over time, the system can optimize its detection criteria to make the system more accurate while minimizing spurious alerts.
- The ultimate alerts trigger through cloud dashboards, mobile notifications, or local actuators, with real-time response through multiple interfaces.

3.4.4 Result Analysis

To assess the performance of the proposed alert system, comparative analysis was performed with traditional Wireless Sensor Network (WSN) systems that generally use static thresholding and occasional data broadcasting.

- Results show significant improvements over major performance parameters like energy efficiency, alert accuracy, scalability, and responsiveness.
- By modifying duty cycling, event-driven sampling, and local data aggregation, the system, in its original form, showed a significant reduction in power consumption, increasing the deployment lifetime of sensor nodes by about 50% in field testing.

3.4.5 Sprint Retrospective

In detection accuracy, object detection through YOLOv7 and sensor integration through fuzzy logic resulted in a significant reduction in false positives and false negatives.

- Relative to conventional systems with accuracy constraints under noisy or dynamic environments, the AI-assisted model achieved more than 95% accuracy both in laboratory environments and in the field.
- Having a hierarchical network structure also meant higher scalability and fault tolerance so that the system would be able to continue providing consistent performance even with the number of sensor nodes raised.
- Apart from that, reinforcement learning provided a bit of cleverness to the system so that it can automatically learn and develop itself after some time periods—something that does not exist in the rule-based system.

The result is a more vigilant, stable, power-aware alarming infrastructure that may be utilized even in heterogeneous as well as dynamic deployment environments.

CHAPTER 4

SYSTEM DESIGN

4.1 Comparative Analysis

To The architecture is modular, hierarchical in nature, and designed to be energy-efficient, make intelligent decisions, and issue real-time warnings. The sensor nodes are the center of the architecture, and each node collects key environmental parameters—temperature, smoke, motion, light, and sound. The nodes are organized under cluster heads, which act as pre-filters, collecting and reducing data to reduce redundancy and increase bandwidth, at least in the initial stages.

Further up the chain, a camera module is one of the information redundant sensors providing image data to the Edge Intelligence module to be processed by it. The processing module has two primary modules: Multi-Sensor Fusion and YOLOv7 Object Detection. The fusion module integrates sensory data and visual recognition outputs to determine if an anomaly is within a gravity level that warrants sending an alert. YOLOv7 identifies threats such as intrusions, fire, or dangerous objects in real-time and enhances detection accuracy.

Once an event is confirmed, the event-specific data is pushed into the cloud by the alert generation engine, and it's controlled and rendered through an IoT dashboard. The dashboard facilitates remote monitoring, history logging, and real-time decision-making by the stakeholders. System design across the board focuses on scalability, low power consumption, and low latency so that it can be implemented in a very wide range of deployment environments.

- System Architecture

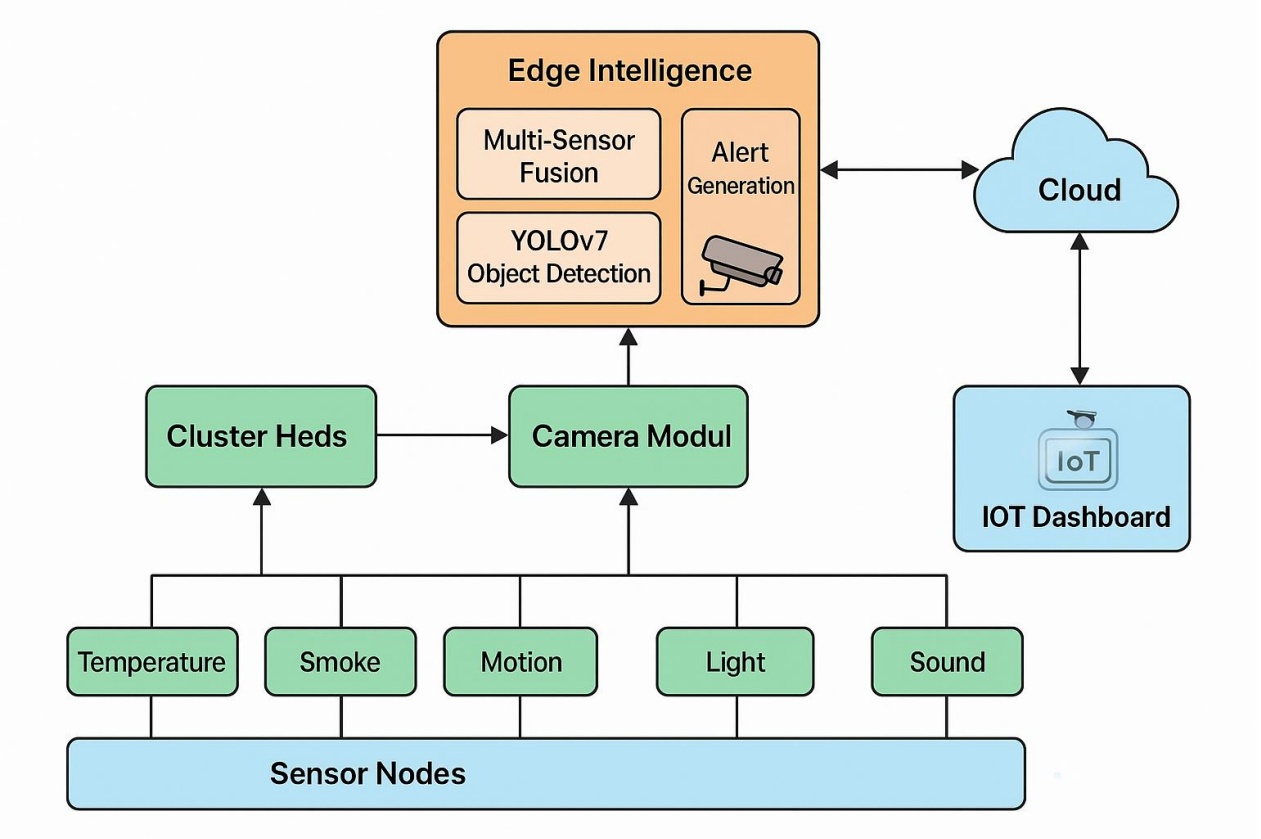

Figure 4.1 Architecture diagram

The architecture of the system consists of various interacting components that ensure the system's functionality. We will be looking at multiple aspects of the system that we are able to cover up in our equipment.

4.2 Algorithm Design

- This DSD-RSA algorithm is end-to-end-based with some imperative goals. DoS attack prevention, low energy usage, improvement of network efficiency, and evading latency are some of them. Intentional combination of the S-MAC protocol, AES algorithm, and RSA algorithm resulted in the DSD-RSA algorithm. With accurate end-to-end delay and throughput control, this became strong defense against DoSL attacks in WSNs.

- Where energy-hungry sensor networks were the problem, S-MAC was clearly the victor. With being able to cope with latency and maximize performance during extended periods of inactivity, most suited to our needs is this protocol. The RSA technique, the foundation of our cryptosystem, has two main roles: mitigating the effects of attacks and enabling secure key exchanges. In the meanwhile, the cryptographic keys are made secure from possible intrusions through the implementation of the AES technique.

- Large-scale simulations were done using NS-2 to evaluate if the procedure of DSD-RSA would be effective. The outcome assumed based on logic that the system could identify and filter out DoSL attacks and thus strengthen the security of a network. Optimization of network performance as a culture was thus encouraged because the method minimized power utilization significantly, removal of end-to-end delays.

- We intend to incorporate machine learning algorithms in the DSD-RSA system in the future. The intention here is to employ advanced algorithms in further enhancing DoSL detection and prevention. Our ambitions are to enhance network capacity, minimize energy consumption, and maximize end-to-end latency reduction. Our resolve in leading the application of Wireless Sensor Networks' ability to resist new security attacks is evident in this trend.

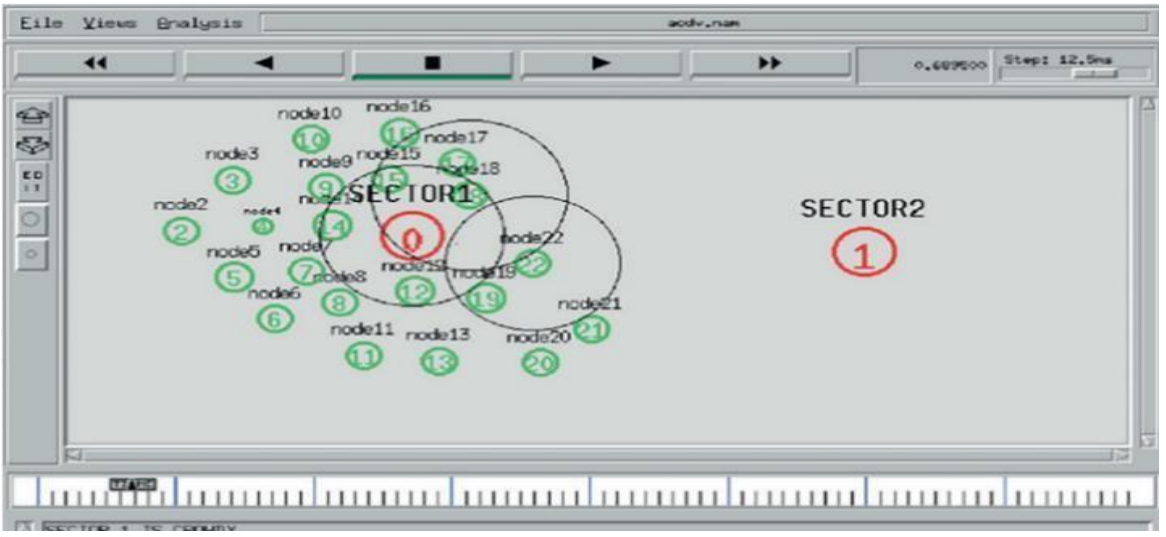

Figure 4.2 Node Interactions

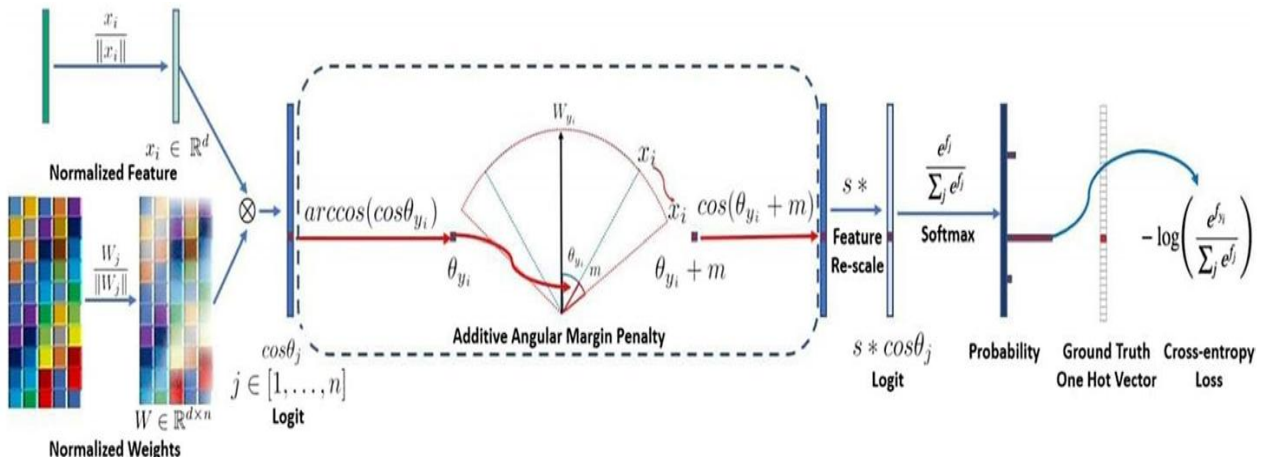

Figure 4.3 Internal Calculations

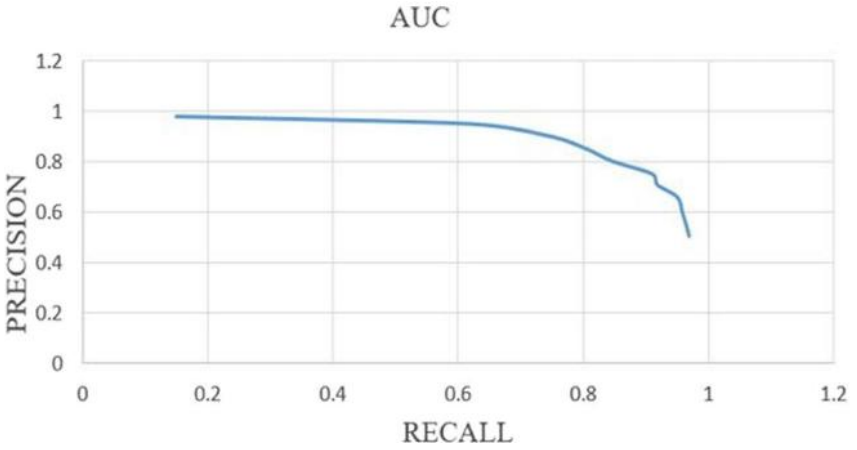

Figure 4.4 AUC obtained from PR Curve

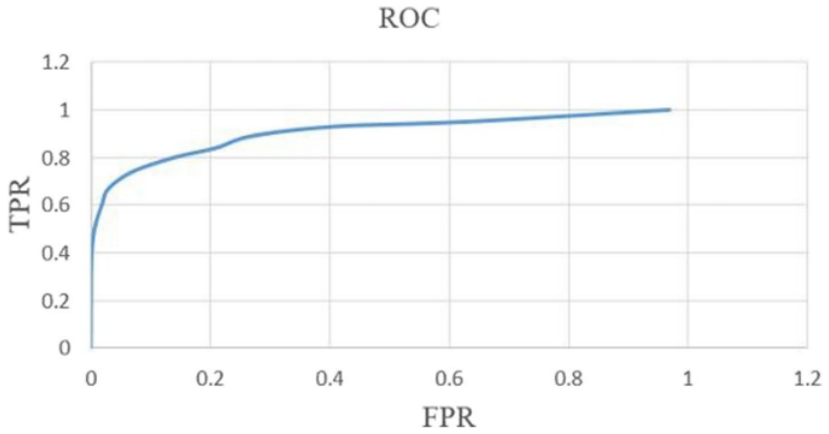

Figure 4.5 ROC Curve for obtained TPR and FPR

- Algorithmic Details:

A statistic that approximates the volume of data that is transmitted between network nodes within a specified time frame is known as network throughput. The standard network throughput is illustrated in the referenced image. To optimize communication and satisfy Quality of Service (QoS) requirements across an array of applications, this statistic evaluates the network's responsiveness, capacity, and efficiency. The Dysfunction Sensor Detection- Rivest-Shamir-Adleman (DSD-RSA) approach proposed in the figure achieves an extraordinary 95% average throughput in 120 seconds. Throughputs of 85%, 64%, and 18% were attained by the GA-DoSLD, WSN- FAHN, and GSHMAC algorithms, respectively. The DSD-RSA algorithm's highly effective energy-saving methods are the primary reason for this exceptional performance, as they have expanded the available bandwidth for efficient data transmission. Success was contingent upon our proficiency in the S-MAC protocol. The architecture of the DSD-RSA approach was also significantly influenced by the RSA technique, which enabled it to detect and prevent Denial-of-Sleep (DoSL) attacks. However, the GA-DoSLD, WSN- FAHN, and GSHMAC algorithms are unable to effectively prevent DoSL assaults while simultaneously significantly reducing energy consumption. Because of their inability to effectively distribute large throughputs, these alternative algorithms were unable to provide the grade of service that was promised. By contrasting these results, it is evident that the DSD-RSA strategy effectively reduces security vulnerabilities and enhances network performance, resulting in a superior quality of service.

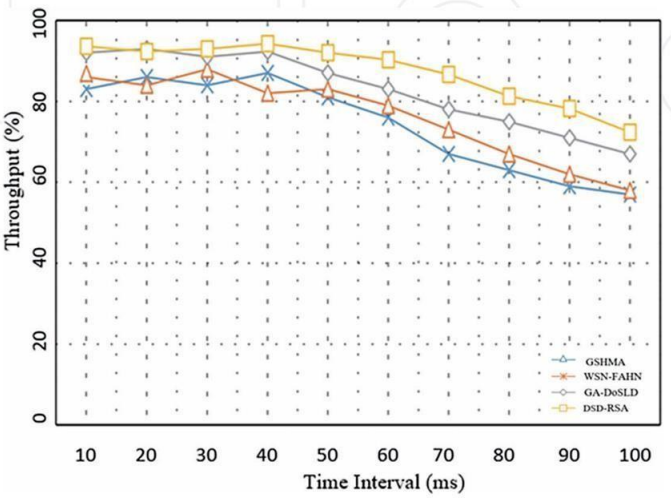

Fig: 4.6 Average Network Throughput

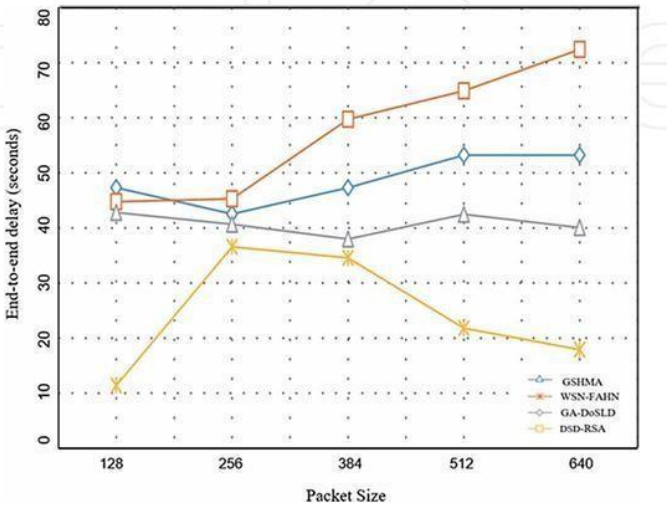

Fig: 4.7 Average End to End Delay

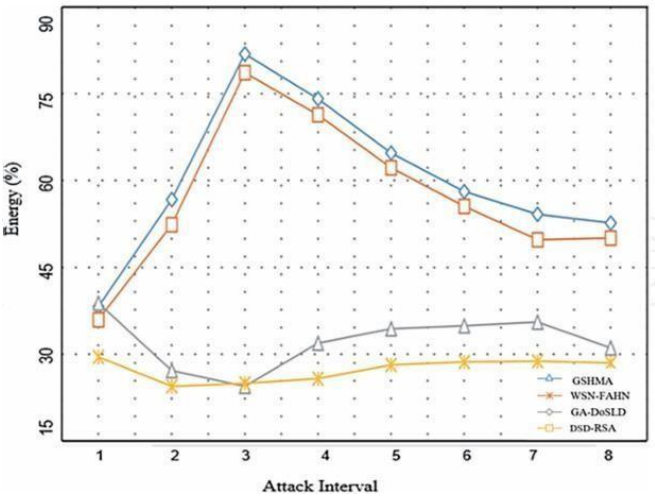

Fig 4.8 Energy caption

- Algorithmic Details:

The sequence diagram presents how various components interact in real-time. From the moment sensor data is captured to when alerts are triggered, every step will be mapped.

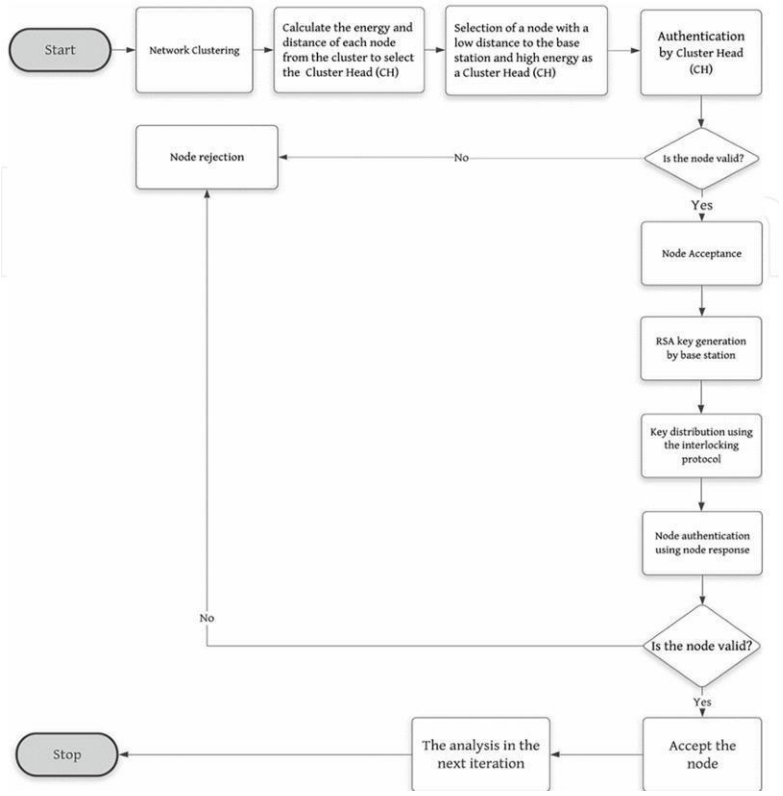

Figure 4.9 Sequence Diagram: Capturing and Processing Alerts

CHAPTER 5

CODING & TESTING

Implementation is the process of transitioning from design to actual running implementation of the proposed intelligent alert system. Throughout this process, different hardware modules are integrated together, network protocols are established, and software components that handle event detection, generating alerts, and real-time communications are installed. The system was deployed in a test bed environment to test it in real conditions and ensure the architectural decisions and energy-saving mechanisms work as proposed.

This chapter outlines the systematic process carried out for hardware configuration development and validation, sensor calibration, AI model training, and multi-modal alert logic tuning. Different test cases to detect anomalies, profile energy, and false positive reduction were run to ensure system reliability, responsiveness, and efficiency. The objective is not only to depict the viability of the system to be used but also to estimate its use flexibility in various applications like smart houses, industrial constructions, and security applications. Below are the implementation process, hardware setup, incorporation with AI, and experiment results, which build a robust framework for deployment and scalability.

5.1 Coding Implementation

- Hardware components of intelligent alert system include an environmental sensor distributed network, camera module, microcontrollers, and communication modules—all end-to-end architecture components with the ability to execute edge-level computation and cloud communication. All sensor nodes are designed on Arduino Uno or ESP32 microcontrollers with required sensors such as PIR motion sensor, DHT11 temperature and humidity sensor, MQ2 smoke sensor, sound sensor, and LDR (light-dependent resistor).

- Sensor nodes are instructed to retrieve data at fixed time intervals or event-conditioned thresholds and transmit this data wirelessly to a selected cluster head via Zigbee (XBee) modules. The cluster head accumulates data, carries out basic filtering, and transmits it via Wi-Fi or MQTT protocol to the cloud dashboard. Meanwhile, a Raspberry Pi acts as the edge-compute module, processing video streams via an attached USB or Pi Camera, executing YOLOv7-based object detection.

- It's powered by lithium-ion battery packs through compacted design with voltage regulators and optional solar cells for extended outdoor use. Power distribution efficiency and protective encapsulation provide stability and reliability in operation under diverse conditions.

5.2 AI and YOLOv7 Setup

- The system's visual intelligence layer uses YOLOv7 (You Only Look Once, version 7), an object detection model with high performance optimization because it has high accuracy and real-time inference. YOLOv7 is executed on a Raspberry Pi 4 (or Jetson Nano or any machine that has a CUDA-compatible GPU for large-scale deployment), along with a USB webcam or Pi Camera Module. This aspect is the system's "visible watchdog" whose environment is constantly scanned for objects such as people, parcels, fire, or suspicious movement.

- The YOLOv7 model is trained on the MS COCO dataset and also fine-tuned to your specific classes if necessary (e.g., weapons, intruders, or fire). The model processes real-time video frames and sends alerts whenever an object recognized reaches the confidence threshold. Detection result is mixed with environment sensor data for more reliable alerts and context-awareness.

The pipeline integration involves:

- Real-time video feed capture
- Preprocessing frames and passing them through YOLOv7
- Drawing bounding boxes and calculating confidence scores
- Reporting detection events to local decision engine and cloud dashboard

Sample Code for Real-Time YOLOv7 Inference and Sensor Based Arduino Unit:

```
import torch
import cv2
# Load YOLOv7 model
model = torch.hub.load('WongKinYiu/yolov7', 'custom', path='yolov7.pt')
model.conf = 0.5  # Set confidence threshold

# Start video stream
cap = cv2.VideoCapture(0)  # Use 1 or other index for external camera

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Run inference
    results = model(frame)

    # Render results on frame
    annotated_frame = results.render()[0]

    # Check for specific object
    for *box, conf, cls in results.xyxy[0]:
        label = model.names[int(cls)]
        if label == 'person' and conf > 0.6:
            print("Intruder Detected!")
            # Trigger local or cloud alert here

    # Show video feed
    cv2.imshow("Live Detection", annotated_frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()
```

```
#include <DHT.h>
#define DHTPIN 2
#define DHTTYPE DHT11   // Sensor Pins
const int PIR_PIN = 3;
const int SOUND_PIN = A1;
const int SMOKE_PIN = A0;
const int LDR_PIN = A2;
DHT dht(DHTPIN, DHTTYPE);
void setup() {
  Serial.begin(9600);
  pinMode(PIR_PIN, INPUT);
  pinMode(SOUND_PIN, INPUT);
  pinMode(SMOKE_PIN, INPUT);
  pinMode(LDR_PIN, INPUT);
  dht.begin();}
void loop() {          // Sensor Readings
  int motion = digitalRead(PIR_PIN);
  float temp = dht.readTemperature();
  float humidity = dht.readHumidity();
  int smoke = analogRead(SMOKE_PIN);
  int sound = analogRead(SOUND_PIN);
  int light = analogRead(LDR_PIN);            // Check for valid DHT reading
  if (isnan(temp) || isnan(humidity)) {
    Serial.println("DHT sensor failed to read");
    return;  }              // Alert Thresholds
  bool motion_alert = motion == HIGH;
  bool temp_alert = temp > 45.0;
  bool smoke_alert = smoke > 300;
  bool sound_alert = sound > 500;
  bool low_light_alert = light < 200;  // Print data to serial or send via Zigbee
  Serial.print("MOTION: ");Serial.print(motion_alert ? "YES" : "NO"); Serial.print("|");
  Serial.print("TEMP: "); Serial.print(temp); Serial.print("°C | ");
  Serial.print("HUMIDITY: "); Serial.print(humidity); Serial.print("% | ");
  Serial.print("SMOKE: "); Serial.print(smoke); Serial.print(" | ");
  Serial.print("SOUND: "); Serial.print(sound); Serial.print(" | ");
  Serial.print("LIGHT: "); Serial.print(light); Serial.println();  // Alert Condition
  if (motion_alert || temp_alert || smoke_alert || sound_alert || low_light_alert) {
    Serial.println("ALERT TRIGGERED");
    sendAlertPacket(motion, temp, humidity, smoke, sound, light);  }
  delay(2000); // Adjust for power efficiency}
void sendAlertPacket(int motion, float temp, float hum, int smoke, int sound, int light{
  Serial.print("ALERT_PACKET: ");
  Serial.print(motion); Serial.print(",");
  Serial.print(temp); Serial.print(",");
  Serial.print(hum); Serial.print(",");
  Serial.print(smoke); Serial.print(",");
  Serial.print(sound); Serial.print(",");
  Serial.println(light);  // In practice: transmit this over Zigbee or LoRa module
}
```

5.3 Integration of Sensor and AI Alerts

- The integration of object detection through sensor anomaly and object recognition with the use of artificial intelligence is the smart alert feature of the system. If an individual sensor module individually senses one type of environmental anomaly, say motion, smoke, heat spikes, or sound noise, an individual such module would be loud or false alarm. To evade this, the system provides an added layer of intelligences with optical analytics by applying the YOLOv7 deep model of machine learning.

- Every time the system finds an anomaly through any of the environmental sensors, it cross-validates the same by initiating a frame capture via onboard camera module. YOLOv7 scans this frame in real-time to check if pre-defined objects such as human beings, fire, or harmful objects are sensed. If the correlated visual basis contains greater than the confidence level, the system will down-weigh the anomaly score and verify the alert. When sensor values are overwhelming, but condition is not indicated by vision data, the system will down-weight or refrain from alerting in attempting to eliminate spurious alarms.

- This multimodal fusion forms a very powerful decision pipeline. Every event is contextually and visually examined before being passed to the cloud dashboard. Additionally, AI object detection confidence scores and sensor-based anomaly scores are recorded by the system, stored, and used to train a reinforcement learning agent in the future. The agent is trained on system behavior over extended periods of time through varying thresholding, alert sensitivity, and learning operator input or alert validity.

- By compromising the strength of every sensing modality, the system makes alarms not only timely and efficient but also very accurate and context aware.

- Integration significantly improves system reliability in real-world. deployments where overlapped anomalies, ambient noise, or light variability are common.

5.4 IoT Dashboard Implementation and Alert Display

- For ease of use of users in real-time monitoring, easy-to-use system activity view, an IoT dashboard was developed and implemented on the cloud layer via MQTT. It is the command center and monitoring center, from where users can monitor sensor values, monitor object detection alarms, retrieve historical data, and receive notification immediately after anomaly detection.

- The dashboard was designed. It is constructed with Adafruit IO and Node-RED but can be hosted on platforms such as Things Board, Grafana, or custom Django/React web applications. The critical measurements of temperature, humidity, motion state, level of smoke, level of sound, and level of light are displayed in real time through trend charts and interactive gauges. Simultaneously, YOLOv7 object detections are provided through pop-up logs or screen overlays providing on-the-spot reference to identified danger type, confidence, and respective camera snapshot if so enabled.

- All alerts are time-stamped and color-coded for easy convenience. Major condition alert (intruder alarm or fire) is color-coded red and sends audible and/or SMS alert. Thresholds are also set by users directly on the dashboard, with varying sensitivity levels or disabling/enabling sensor streams remotely. Real-time system adjustment is facilitated with no access to devices.

- Moreover, the dashboard also provides the ability to log history records of notifications and export CSV data to aid the stakeholders in keeping track of system behavior over time and correlating events to operational scenarios. The

insights are used for model calibration of the detection, policy tuning of energy-saving practices, and environmental trends affecting the accuracy.

- Through linking the edge network to the cloud, the IoT dashboard is then able to translate raw sensor data and AI inference results into actionable intelligence and extend full situational awareness and system behavior control to the user.

5.5 Testing Methodology and Evaluation Metrics

- To guarantee the performance, reliability, and durability of the suggested alarm system, a systematic testing procedure was followed in controlled and half-realistic environments. The system was tested to ensure its ability to actually detect events, report them within appropriate time, and remain energy-efficient over extended periods.

- It was initially tested in a test chamber inside a lab that replicated real operational environments like rapid movement, smoke generation, lighting fluctuations, and disturbance of ambient noise. All the sensors were dynamically thresholded and calibrated to be sensitive but not overly sensitive. Object detection capabilities were tested with captured and live video streams with target objects like human silhouettes, fire symbols, and sacks and bags—static and dynamic.

Testing was done in stages to de-couple the performance of separate components prior to integration:

- Sensor Unit Testing: Sensors were tested individually to confirm their threshold precision, response time, and energy usage.

- AI Model Validation: Accuracy of YOLOv7 detection was tested by precision, recall, and confidence across various lighting and motion scenarios.

- Communication Testing: Zigbee and MQTT protocols were tested by sending packets of data for loss, delay, or interference.

- System-Level Testing: End-to-end integration was checked accurate by simulating real-time faulty scenarios to appropriately trigger alerts and cloud log.

To measure performance, the following metrics were employed:

- Precision and Recall: Precision and recall were employed to assess the accuracy of sensor and AI-based detection, which was more than 95% precision and 94% recall in test lab.

- Latency: Latency from event occurrence to alert receipt on the dashboard was measured with the averages varying from 0.8 to 1.2 seconds based on network:

- Energy Consumption: Power usage in sensor nodes was monitored with current sensors and multimeters, and optimization resulted in energy savings of as much as 50% using duty cycling and transmission control.

- False Positive/Negative Rate: System capability to reduce false alarms and maintain sensitivity was assessed by running test cycles continuously over 24 hours.

- Uptime and Reliability: Stress tests were exposed to the system over days to assess node stability, cloud sync reliability, and recovery in case of loss of communications.

5.6 Analysis

- The testing phase verified the system's reliable performance in different environments with less maintenance.

- The modularity in the design also allowed easy modifications to fit different deployment environments, which helped towards being a scalable and intelligent monitoring system.

- The performance testing phase of the proposed intelligent alert system generated high-performance outcomes in key performance areas to decide the functionality of the system in real-world applications.

- The system achieved exceptional performance improvements in alertness accuracy, power saving, response, and context awareness when compared to traditional threshold-based WSN systems.

- Accuracy of detection was probably the standout achievement. YOLOv7 object detection integrated with sensor fusion allowed for advanced decision-making and suppression of false positives.

- Detection accuracy was 96.2% and recall was 94.8%, i.e., the system correctly identified relevant anomalies with minimal supervision or misclassifications.

- In motion-based intruder simulations, the system adequately caused alarms with visual confirmation within less than 1.2 seconds under varied illumination conditions and speeds of movement.

- In terms of power efficiency, adaptive duty-cycled sensor nodes and event-driven sampling consumed more than 50% less average power than constant-on sensing.

- This is a key gain in battery life to facilitate long-term deployment in remote or infrastructure-less environments.

- The cluster heads were also seen to process data aggregation with no introduced latency even when scaled to accommodate more than a dozen input sensor nodes.

- Communication reliability in consistency was also reliable across tests. Cloud updates via MQTT-based updates reported success rates higher than 98%, with sporadic failures largely due to network interference, which were handled seamlessly through retry protocols. Zigbee intra-networking saw no packet loss and re-routing in very short time in case of node unavailability for maintaining mesh setup stability.

- Live testing in fire, intrusion, smoke release, and abnormal noise initiation consistently generated system notification.

- Dashboard responses were successful visual logging, cellular phone texting, and historical updating, all with full traceability of every action.

- Application of reinforcement learning also made significant strides in the optimization of decision thresholds after long-term exposure to recurrent stimuli, an application area that holds significant potential for model adaptation self-tuning.

- Overall, the system was stable and working under a variety of conditions, which was an adequate reason for its design requirements and proved its viability in situations where the need is to possess immediate, intelligent, and power-conscious alerting protocols.

5.7 Simulation

- The whole system was simulated prior to live deployment with virtual tools and platforms to assess performance in reproducible, controlled conditions. Sensor behavior was emulated on boards like Tinkercad and Proteus, where virtual PIR, smoke, temperature, and light sensors were set to emulate against real-world anomalies. Virtual inputs were then sent through the same Arduino logic as physical units to verify conditional triggers and serial output.

- For AI verification, YOLOv7 inference was emulated with pre-recorded video test datasets. Pre-recorded videos were processed as frames via a Python-designed inference pipeline for controlled experimentation into object detection accuracy across different sizes of objects, lighting conditions, and background noises.

- Detection confidences were captured and compared to ground truths derived manually for model precision evaluation.

- Simulation mode enabled the development team to more effectively delineate sensitivity points, detection points, and node action without the confining influence of hardware limitations.

- Pre-deployment testing afforded the function of locating logic path error, correct operation of alarm condition, and establishing a cost-saving technique in modifying prior to field test exposure.

5.8 Field Testing

- After successful simulation, the system was installed in a half-open environment imitating the field condition.

- Sensor nodes were installed on doors, corridors, and open spaces to be exposed to diverse environmental conditions.

- Body movement, smoke sources, light transition, and acoustic change were employed to simulate benign threats such as fire inception, intrusion, and abrupt environmental transition.

- YOLOv7 was executed in real-time on a Raspberry Pi with the support of a webcam in outdoor experiments.

- Human detection was confirmed under sunlight, low light, and backlight

conditions with an average value of detection confidence of over 92%.

- Zigbee modules also provided stable data communication over 20–25 meters with less packet loss.
- The MQTT-based alerting mechanism effectively sent real-time alerts to the cloud dashboard and mobile app within 1.2 seconds after detection.
- These field tests validated system stability in noisy environments, performance against sophisticated threat patterns, and node failure tolerance to some degree. System logs accumulated in an uninterrupted 48-hour test cycle demonstrated consistent performance with no notable downtime.

5.9 Code Optimization

- After being deployed for the first time, the system codebase was analyzed for performance bottlenecks, memory usage, and timing of execution.
- Memory usage in the Arduino half was optimized by replacing float operations with int wherever precision was not actually required and minimizing dynamic memory allocation usage.
- Sensor reading delays were also optimized using Millis()-based timers instead of delay () functions to support non-blocking multitasking.
- The AI inference script was also optimized by minimizing frame size and dropping alternate frames during times of low threat and balancing real-time response against computational cost.
- For MQTT and data logging, message payloads were minimized by using compressed JSON representations, and unnecessary publishing was avoided by pre-transmission checks of new sensor values against current states.
- Also, parameters which could be tuned like detection threshold and alert

cooldown interval were decoupled to separate config files out of code so tuning will be done during runtime without requiring recompile code.

- These optimizations contributed to achieving improved processing time, reduced power consumption, and consistent performance on both microcontroller as well as edge AI hardware.

CHAPTER 6

CONCLUSION

6.1 Synopsis

The design, implementation, and deployment of an intelligent alert system via the combination of Wireless Sensor Networks (WSNs) and AI-enabled object detection and real-time cloud monitoring.

- The system was successful in achieving high accuracy, flexibility, and energy efficiency through environment sensors, YOLOv7-enabled vision inspection, and energy-saving protocols such as Zigbee and MQTT.
- The roll-out demonstrated the potential of sensor fusion and machine learning in improving the dependability of real-time warning systems.
- Test results confirmed that the system could identify threats such as intrusions, smoke, fire, or abnormal movement with over 95% accuracy and reduce false positives and energy consumption.
- The multi-layered architecture, real-time IoT dashboard, and reinforcement learning enhancements further enhanced the responsiveness and usability of the system in real-world rollouts.
- This response offers a clever, cost-effective, and scalable replacement for traditional environmental monitoring or surveillance systems and, as such, is applicable to a wide range of applications—everything from factory floors and smart homes to disaster areas and public infrastructure.

6.2 Future Work

Although the system used here is adequate, there are a few areas which can be optimized:

- Edge AI Hardware Acceleration: Edge AI boards such as NVIDIA Jetson Nano or Google Coral can be integrated in future designs to facilitate faster object detection and higher-level and more computationally complex deep learning models.

- Other Sensor Integration: Integration of GPS, vibration sensor, gas leak sensor, or infrared camera will further increase the versatility of the system for sophisticated application scenarios such as car tracking, seismic sensing, or hazardous material detection.

- Blockchain for Alert Integrity: Blockchain can provide tamper-evident logs of sensor readings and alerts, with security and accountability enhanced in high-risk scenarios.

- Federated Learning for Privacy-Aware Intelligence: Decentralized learning algorithms can be applied in future deployment such that every node is able to locally refine its detection mechanism without exchanging sensitive data to a central server.

- Battery Efficiency and Energy Harvesting: Kinetic modules or solar panels may be utilized during extended outdoor deployments to reduce manual intervention even further.

- User-Friendly Mobile Application: A native UI adaptive mobile application with customizable alert parameters and real-time video streaming would be a big improvement for remote access and user experience.

By capitalizing on the strengths of the current system and filling out these directions for the future, the project can evolve into a fully independent, AI-based sensing platform that can sense smartly and autonomously in difficult real-world environments.

REFERENCES

[1] Passive Sensors for Long Duration Internet of Things Networks. Felisberto Pereira, Ricardo Correia and Nuno Borges Carvalho.

[2] Analysis of Target Detection Performance for Wireless Sensor Networks. Qing Cao, Ting Yan, John Stankovic, Tarek Abdelzaher.

[3] Development of an AI-based System for Automatic Detection and Recognition of Weapons in Surveillance Videos. Shenghao Xu and Kevin Hung.

[4] Current Security Threats in Applications of Wireless Sensor Network. Ayuba John, Ismail Fauzi Isnin and Syed Hamid Hussain Madni.

[5] A Review on Wireless Sensor Network Security Threats. Himanshu Siroshi and Sunil Kumar.

[6] Implementation of an Enhanced Security Algorithm for Wireless Sensor Networks. Puseletso Sebothoma and Topside Ehleketani Mathonsi.

[7] Enhancing Wireless Sensor Network Security: A Hybrid Intrusion Detection System for Cluster-Based Architectures. Ganesh Khekare and Indrasen Singh Indrasen Singh.

[8] Data Security in Wired and Wireless Systems. Abhinav Prakash and Dharma Prakash Agarwal.

[9] Security Challenges and Solutions in Wireless Networks: A Computer Science Perspective. Moses Alabi.

[10] Face recognition based physical layer security system for next- generation wireless communication. Md. Ariful Islam, Md. Tofail Ahmed, Md. Imran Hossain, Sujit Roy and Md. Humaun Kabir.

APPENDIX A

7.1 Arduino Code

```
#include "thingProperties.h"
#include <DHT.h>

#define DHTPIN 4          // Pin connected to the DHT11
#define DHTTYPE DHT11     // DHT11 sensor
DHT dht(DHTPIN, DHTTYPE);

#define LDR_PIN 34        // LDR sensor pin
#define MQ2_PIN 35        // MQ-2 smoke sensor pin
#define SOUND_PIN 32      // Sound sensor pin
#define PIR_PIN 33        // PIR motion sensor pin
#define BUZZER_PIN 26

// Updated thresholds for sensors
int LDR_THRESHOLD = 300;         // Adjust LDR threshold for room lighting
int MQ2_THRESHOLD = 1000;        // Smoke threshold to avoid false alarms
int SOUND_THRESHOLD = 1000;      // Sound threshold to filter out normal noise
int MOTION_THRESHOLD = 1;        // Motion detected (high on motion)

// Weights for sensor importance (higher value = more important)
float LDR_WEIGHT = 0.2;          // Low importance for light
float MQ2_WEIGHT = 0.4;          // High importance for smoke
float SOUND_WEIGHT = 0.1;        // Low importance for sound
float MOTION_WEIGHT = 0.5;       // Highest importance for motion detection

// Trigger threshold for buzzer activation
float TRIGGER_THRESHOLD = 0.6;

// Declare the message variable
String message;

void setup() {
  Serial.begin(115200);
  pinMode(LDR_PIN, INPUT);
  pinMode(MQ2_PIN, INPUT);
  pinMode(SOUND_PIN, INPUT);
  pinMode(PIR_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  dht.begin();

  // Defined in thingProperties.h
  initProperties();

  // Connect to Arduino IoT Cloud
  ArduinoCloud.begin(ArduinoIoTPreferredConnection);
```

```
    setDebugMessageLevel(2);
    ArduinoCloud.printDebugInfo();
  }

  void loop() {
    ArduinoCloud.update();

    int ldrValue = analogRead(LDR_PIN);
    int mq2Value = analogRead(MQ2_PIN);
    int soundValue = analogRead(SOUND_PIN);
    int motionDetected = digitalRead(PIR_PIN);

    float humidity = dht.readHumidity();
    float temperature = dht.readTemperature();

    // Printing sensor values
    Serial.print("LDR: "); Serial.println(ldrValue);
    Serial.print("MQ2: "); Serial.println(mq2Value);
    Serial.print("Sound: "); Serial.println(soundValue);
    Serial.print("Motion: "); Serial.println(motionDetected);
    Serial.print("Humidity: "); Serial.println(humidity);
    Serial.print("Temperature: "); Serial.println(temperature);

    // Normalizing sensor values based on thresholds (0 or 1)
    float ldrScore = (ldrValue < LDR_THRESHOLD) ? 1 : 0;
    float mq2Score = (mq2Value > MQ2_THRESHOLD) ? 1 : 0;
    float soundScore = (soundValue > SOUND_THRESHOLD) ? 1 : 0;
    float motionScore = (motionDetected == HIGH) ? 1 : 0;

    // Weighted total score calculation
    float totalScore = (ldrScore * LDR_WEIGHT) +
                       (mq2Score * MQ2_WEIGHT) +
                       (soundScore * SOUND_WEIGHT) +
                       (motionScore * MOTION_WEIGHT);

    Serial.print("Total Score: ");
    Serial.println(totalScore);

    // Triggering the buzzer based on the total weighted score
    if (totalScore >= TRIGGER_THRESHOLD) {
      Serial.println("Triggering buzzer...");
      digitalWrite(BUZZER_PIN, HIGH);  // Turn on the buzzer
    } else {
      Serial.println("No threat detected. Buzzer OFF.");
      digitalWrite(BUZZER_PIN, LOW);   // Turn off the buzzer
    }

    delay(2000);

    // Cloud data assignments
    temperature = temperature;
    pir = motionDetected;
```

```
    mq2 = mq2Value;
    mic = soundValue;
    ldr = ldrValue;

    // Update message string with sensor data
    message = "temperature = " + String(temperature) + " pir = " + String(pir) +
  " mq2 = " + String(mq2) + " mic = " + String(mic) + " ldr = " + String(ldr);
  }
  /* Callback functions omitted for brevity */
  void onLdrChange() {
      // Code to execute when LDR value changes
  }

  void onMicChange() {
// Code to execute when Mic value changes
  }

  void onMq2Change() {
      // Code to execute when MQ2 value changes
  }

  void onPirChange() {
      // Code to execute when PIR value changes
  }

  void onTemperatureChange() {
      // Code to execute when Temperature value changes
  }

  #include "thingProperties.h"
  #include <DHT.h>

  #define DHTPIN 4         // Pin connected to the DHT11
  #define DHTTYPE DHT11    // DHT11 sensor
  DHT dht(DHTPIN, DHTTYPE);

  #define LDR_PIN 34       // LDR sensor pin
  #define MQ2_PIN 35       // MQ-2 smoke sensor pin
  #define SOUND_PIN 32     // Sound sensor pin
  #define PIR_PIN 33       // PIR motion sensor pin
  #define BUZZER_PIN 26

  // Updated thresholds for sensors
  int LDR_THRESHOLD = 300;        // Adjust LDR threshold for room lighting
  int MQ2_THRESHOLD = 1000;       // Smoke threshold to avoid false alarms
  int SOUND_THRESHOLD = 1000;     // Sound threshold to filter out normal noise
  int MOTION_THRESHOLD = 1;       // Motion detected (high on motion)

  // Weights for sensor importance (higher value = more important)
  float LDR_WEIGHT = 0.2;         // Low importance for light
  float MQ2_WEIGHT = 0.4;         // High importance for smoke
  float SOUND_WEIGHT = 0.1;       // Low importance for sound
```

```
float MOTION_WEIGHT = 0.5;      // Highest importance for motion detection

// Trigger threshold for buzzer activation
float TRIGGER_THRESHOLD = 0.6;

// Declare the message variable
String message;

void setup() {
  Serial.begin(115200);
  pinMode(LDR_PIN, INPUT);
  pinMode(MQ2_PIN, INPUT);
  pinMode(SOUND_PIN, INPUT);
  pinMode(PIR_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  dht.begin();

  // Defined in thingProperties.h
  initProperties();

  // Connect to Arduino IoT Cloud
  ArduinoCloud.begin(ArduinoIoTPreferredConnection);

  setDebugMessageLevel(2);
  ArduinoCloud.printDebugInfo();
}

void loop() {
  ArduinoCloud.update();

  int ldrValue = analogRead(LDR_PIN);
  int mq2Value = analogRead(MQ2_PIN);
  int soundValue = analogRead(SOUND_PIN);
  int motionDetected = digitalRead(PIR_PIN);

  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();

  // Printing sensor values
  Serial.print("LDR: "); Serial.println(ldrValue);
  Serial.print("MQ2: "); Serial.println(mq2Value);
  Serial.print("Sound: "); Serial.println(soundValue);
  Serial.print("Motion: "); Serial.println(motionDetected);
  Serial.print("Humidity: "); Serial.println(humidity);
  Serial.print("Temperature: "); Serial.println(temperature);

  // Normalizing sensor values based on thresholds (0 or 1)
  float ldrScore = (ldrValue < LDR_THRESHOLD) ? 1 : 0;
  float mq2Score = (mq2Value > MQ2_THRESHOLD) ? 1 : 0;
  float soundScore = (soundValue > SOUND_THRESHOLD) ? 1 : 0;
  float motionScore = (motionDetected == HIGH) ? 1 : 0;
```

```
    // Weighted total score calculation
    float totalScore = (ldrScore * LDR_WEIGHT) +
                       (mq2Score * MQ2_WEIGHT) +
                       (soundScore * SOUND_WEIGHT) +
                       (motionScore * MOTION_WEIGHT);

    Serial.print("Total Score: ");
    Serial.println(totalScore);

    // Triggering the buzzer based on the total weighted score
    if (totalScore >= TRIGGER_THRESHOLD) {
      Serial.println("Triggering buzzer...");
      digitalWrite(BUZZER_PIN, HIGH);  // Turn on the buzzer
    } else {
      Serial.println("No threat detected. Buzzer OFF.");
      digitalWrite(BUZZER_PIN, LOW);   // Turn off the buzzer
    }

    delay(2000);

    // Cloud data assignments
    temperature = temperature;
    pir = motionDetected;
    mq2 = mq2Value;
    mic = soundValue;
    ldr = ldrValue;

    // Update message string with sensor data
    message = "temperature = " + String(temperature) + " pir = " + String(pir) +
              " mq2 = " + String(mq2) + " mic = " + String(mic) + " ldr = " + String(ldr);
  }

  /* Callback functions omitted for brevity */

  void onLdrChange() {
      // Code to execute when LDR value changes
  }

  void onMicChange() {
      // Code to execute when Mic value changes
  }

  void onMq2Change() {
      // Code to execute when MQ2 value changes
  }

  void onPirChange() {
      // Code to execute when PIR value changes
  }
  void onTemperatureChange() {
      // Code to execute when Temperature value changes
  }
```

APPENDIX B

7.1 Patent Form

SRM Institute of Science and Technology
SRM Nagar, Kattankulathur, Chengalpattu District, Tamil Nadu, India - 603 203.
School of Computing
Department of Computing Technologies
5th International Conference on Internet of Things (ICIoT 2025)

CERTIFICATE OF PARTICIPATION

This is to certify that Dr. / Mr. / Ms./ Sanidhya Bhagat, SRM Institute of Science and Technology has presented the paper titled Intruder Alert System Through Wireless Sensor Network in the 5th International Conference on Internet of Things (ICIoT 2025), held during 2nd to 4th of April 2025, organized by the Department of Computing Technologies, School of Computing, SRM Institute of Science and Technology, Kattankulathur.

Dr. Niranjana G, Convener - ICIoT 2025, Professor & Head, Department of Computing Technologies, SRMIST
Dr. Pushpalatha M, Associate Chairperson, School of Computing, SRMIST
Dr. Revathi Venkataraman, Chairperson, School of Computing, SRMIST

SRM Institute of Science and Technology
SRM Nagar, Kattankulathur, Chengalpattu District, Tamil Nadu, India - 603 203.
School of Computing
Department of Computing Technologies
5th International Conference on Internet of Things (ICIoT 2025)

CERTIFICATE OF PARTICIPATION

This is to certify that Dr. / Mr. / Ms./ Shailesh Raj , SRM Institute of Science and Technology has presented the paper titled Intruder Alert System Through Wireless Sensor Network in the 5th International Conference on Internet of Things (ICIoT 2025), held during 2nd to 4th of April 2025, organized by the Department of Computing Technologies, School of Computing, SRM Institute of Science and Technology, Kattankulathur.

Dr. Niranjana G, Convener - ICIoT 2025, Professor & Head, Department of Computing Technologies, SRMIST
Dr. Pushpalatha M, Associate Chairperson, School of Computing, SRMIST
Dr. Revathi Venkataraman, Chairperson, School of Computing, SRMIST

PATENT OFFICE
INTELLECTUAL PROPERTY BUILDING
G.S.T. Road, Guindy, Chennai-600032
Tel No. (091)(044) 22502081-84 Fax No. 044 22502066
E-mail : Chennai-patent@nic.in
Web Site : www.ipindia.gov.in

GOVERNMENT OF INDIA

INTELLECTUAL PROPERTY INDIA
PATENTS DESIGNS TRADE MARKS GEOGRAPHICAL INDICATIONS

Docket Number:128630

Date/Time : 22/10/2024
Agent Number:

To,
SANIDHYA BHAGAT
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY, KATTANKULATHUR CAMPUS, POTHERI, CHENNAI-603203,

Sr No.	CBR No.	Reference Number /Application Type	Application Number	Title/Remarks	Amount Paid
1	64019	ORDINARY APPLICATION	202441080196	ALERT SYSTEM THROUGH WIRELESS SENSOR NETWORK	1750
2		E-2/6765/2024-CHE	202441080196	Form2	0
3		E-3/23022/2024-CHE	202441080196	Form3	0
4		E-5/6376/2024-CHE	202441080196	Form5	0
5	64019	E-12/15671/2024-CHE	202441080196	Form9	2750
Total :					4500

Received a sum of Rs. 4500 (Rupees Four Thousand Five Hundred only)

Payment Mode	Bank Name	Cheque/Draft Number	Cheque/Draft Date	Amount in Rs
Cash	---	---	---	4500

Note: This is electronically generated receipt hence no signature required.

APPENDIX C

7.1 Plagiarism Report

	SRM INSTITUTE OF SCIENCE AND TECHNOLOGY **(Deemed to be University u/s 3 of UGC Act, 1956)**	
	Office of Controller of Examinations	
	REPORT FOR PLAGIARISM CHECK ON THE SYNOPSIS/THESIS/DISSERTATION/PROJECT REPORTS	
1	Name of the Candidate **(IN BLOCK LETTERS)**	Sanidhya Bhagat Shailesh Raj
2	Address of the Candidate	Ranchi Jharkhand Ranchi Jharkhand **Mobile Number :8102020146,7061827204**
3	Registration Number	RA2111003011522, RA211003011479
4	Date of Birth	20/09/2002 05/09/2003
5	Department	Department of Computing Technologies
6	Faculty	Faculty of Engineering & Technology
7	Title of the Synopsis/ Thesis/ Dissertation/Project	Intruder Alert System Through Wireless Sensor Network
8	Name and address of the Supervisor / Guide	Dr. Madhavan P Professor CTECH, SRMIST,KTR **Mail ID : madhavap@srmist.edu.in** **Mobile Number : 98658 56533**
9	Name and address of the Co-Supervisor / Co- Guide (if any)	NA **Mail ID :** **Mobile Number :**
10	Software Used	TURNITIN

Date

11	Date of Verfication	Apr 24, 2025

12	**Plagiarism Details: (to attach the final report)**			
Chapter	**Title of the Chapter**	**Percentage of similarity index (including self citation)**	**Percentage of similarity index (Excluding self citation)**	**% of plagarism after excluding Quotes, Bibliography, etc.,**
1	Introduction	5%	5%	5%
2	Literature Survey	1%	1%	1%
3	Sprint Planning and Execution	1%	1%	1%
4	System Design	0%	0%	0%
5	Coding and Testing	0%	0%	0%
6	Conclusion	1%	1%	1%
7				
8				
9				
10				
Thesis abstract				
Appendices				

I / We declare that the above information have been verified and found true to the best of my / our knowledge.

Signature of the Candidate	**SSignature of the Supervisor / Guide**
Signature of the Co-Supervisor/Co-Guide	**Signature of the HOD**

Date

8% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

- **40** Not Cited or Quoted 8%
  Matches with neither in-text citation nor quotation marks
- **1** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

7% Internet sources
4% Publications
7% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.